

```
---
title: Encryption Request Flow
sidebar_position: 1
---
```

Encryption Request Flow

Overview

This document outlines the complete flow of the encryption request process using CoFHE.js, a TypeScript library designed to help users encrypt data for private computation with smart contracts. Understanding this process is essential for developers who want to enable their users to interact with privacy-preserving smart contracts using encrypted inputs.

Key Components

Component	Description
<code>**dApp**</code>	The decentralized application that interacts with the user and the contracts
<code>**CoFHE.js**</code>	TypeScript package designed for seamless interaction Fhenix's co-processor
<code>**Threshold Network**</code>	(When applicable) Handles secure decryption operations

Flow Diagram

The following diagram illustrates the complete flow of an Encryption request in the CoFHE ecosystem:

```
[[Diagram](../../static/img/assets/Encrypt%20a%20value.svg)]
(../../static/img/assets/Encrypt%20a%20value.svg)
*Figure 1: End-to-end flow of an Encryption request through the CoFHE system components*
```

Step-by-Step Flow

1. `**Install, include and initialize CoFHE.js in your project (full details [here](/docs/devdocs/quick-start/getting-started.md))**`

```
```bash
npm install cofhejs
```
```javascript
const { cofhejs, FheTypes, Encryptable } = require("cofhejs/node");
```
```

2. ****Data preparation****

The data is encrypted locally using the `encrypt` function.
Under the hood, `encrypt` encrypts the data using the TFHE library and create a zkPoK to prove the encryption is correct.

3. ****Encryption verification****

The zkPoK is verified using the `verify` function.
This verification process ensures that the ciphertext was generated correctly—that it represents a valid encryption of a known plaintext—and that the data has not been tampered with. Upon successful verification, the encrypted data is stored in the Data Availability (DA) layer.

The function returns a value handle that can be used to reference the encrypted data later, along with a signature.

5. ****Using the encrypted data****

The user can send the value handle to the contract as an encrypted input. This handle represents the ciphertext stored in the DA layer and allows the contract to reference the encrypted value

Note: Read more about the implementation details [here](/docs/devdocs/cofhejs/encryption-operations)

title: Decryption Request Flow
sidebar_position: 3

Decryption Request Flow

The process of requesting decryption through Smart Contracts starts the same as every other [FHE Operation Request](fhe-operation.md)  steps 1-4 \

Here we'll continue from FheOS server handling such request as follows:

Flow Diagram

The following diagram illustrates the complete flow of an FHE Decryption request in the CoFHE ecosystem:

[![Diagram](../../../../../static/img/assets/Decryption%20Transactions.svg)](../../../../../static/img/assets/Decryption%20Transactions.svg)

Figure 1: End-to-end flow of an FHE Decryption request through the CoFHE system components

 [Step 1-4](fhe-operation.md)
>![Bullet](../../../../../static/img/assets/1.png) ![Bullet](../../../../../static/img/assets/2.png) ![Bullet](../../../../../static/img/assets/

3.png) ![Bullet](../../../../static/img/assets/4.png) ![Bullet]
(../../../../static/img/assets/5.png) ![Bullet](../../../../static/
img/assets/6.png)

📌 Step 5: FheOS server – Decryption Execution
The FheOS server handles decryption requests:

1. ****Create execution thread**** on the fheOS server
2. ****FheOS server calls the threshold network**** with:
 - The ciphertext to be decrypted
 - Transaction hash from the host chain
 - Original operation handle

📌 Step 6: Threshold network security protocol

- Verify the host chain requested the desired decryption
- Retrieve the actual ciphertext hash from private storage
- Validate ciphertext hash integrity
- Perform secure decryption

📌 Step 7: FheOS Notifies the Aggregator with the decrypt result

- Call appropriate callback function on the Aggregator
- The Aggregator calls the TaskManager with relevant result

📌 Step 8: TaskManager emit event with decryption result

- Provide decrypted result by emitting an event `DecryptionResult`
- The event consists of `ciphertext handle`, `result`, `requestor`
(of that decrypt operation)

title: FHE Operation Request Flow
sidebar_position: 2

FHE Operation Request Flow

Overview

This document outlines the complete flow of an FHE (Fully Homomorphic Encryption) operation request in the CoFHE ecosystem through Smart Contracts. Understanding this process is essential for developers integrating private computation capabilities into their smart contracts.

Key Components

| Component | Description |
|------------|---|
| ***dApp*** | The decentralized application that requests FHE |

```

operations |
| **FHE.sol** | The library providing FHE operation functions |
| **Task Manager** | Verifies and forwards operation requests |
| **Aggregator** | Manages the request queue and communicates with the
execution layer |
| **fheOS Server** | Executes the actual FHE operations |
| **Threshold Network** | (When applicable) Handles secure decryption
operations |

```

Flow Diagram

The following diagram illustrates the complete flow of an FHE operation request in the CoFHE ecosystem:

```

[![Diagram](../../static/img/assets/Transactions.svg)]
(../../static/img/assets/Transactions.svg)

```

Figure 1: End-to-end flow of an FHE operation request through the CoFHE system components

Step-by-Step Flow

📌 Step 1: Integration with CoFHE.js

1. The decentralized application (dApp) integrates with CoFHE by utilizing ****Cofhe.js**** for encryption.
[See in GitHub](<https://github.com/FhenixProtocol/cofhe.js>) ![Bullet](../../static/img/assets/1.png)

2. [Encrypt request](./encryption-request.md) using CoFHE.js, returns `InEuInt` structure.

> 📝 ****Note:**** This step happens on the client side before blockchain interaction.

📌 Step 2: Requesting an FHE Operation

When the dApp needs to perform an encrypted operation within the smart contract: ![Bullet](../../static/img/assets/2.png)

1. ****Import the FHE library in Solidity****:

```

```solidity
import "@fhenixprotocol/cofhe-contracts/FHE.sol";
```

```

2. ****Call the appropriate FHE function**** from the imported library:

```

```solidity
// using trivial encrypt or the returned structures from the
previous step.

```

```

function addExample(InEuInt32 encryptedInput) {
 euInt32 lhs = FHE.asEuInt32(encryptedInput);

```

```

 uint32 rhs = FHE.asEuint32(10);

 // Request an operation (addition in this example)
 uint32 result = FHE.add(lhs, rhs);
}
...

```

3. **FHE.sol forwards the request** to the Task Manager contract

### ### 📌 Step 3: Task Manager Processing

The Task Manager serves as the gateway for all FHE operation requests:

1. **Validate request structure** to ensure all inputs are properly formatted
2. **Verify access permissions** by checking if the caller has proper access to the encrypted inputs (using ACL.sol)  (.../.../.../static/img/assets/3.png)
3. **Generate a unique handle** that will be used to reference the future ciphertext result
4. **Return the handle** to the calling dApp contract
5. **Emit an event** containing the operation details for the Aggregator to process  (.../.../.../static/img/assets/4.png)

### ### 📌 Step 4: Aggregator Processing

The Aggregator manages the queue of FHE operation requests:

1. **Listen for events** from the Task Manager  (.../.../.../static/img/assets/5.png)
2. **Add the request** to a processing queue
3. **Forward request details** to the fheOS server  (.../.../.../static/img/assets/6.png)
4. **Track the request status** throughout its lifecycle

### ### 📌 Step 5: FheOS server – FHE Operation Execution

The FheOS server handles requests:

1. **Create execution thread** on the fheOS server
2. **Execute the requested operation** on encrypted data
3. **Generate result ciphertext** containing the encrypted result
4. **Map the handle** to the actual ciphertext hash in the private storage
5. **Make result available** for subsequent operations
6. **Notify the Aggregator** of operation completion

### ### 📌 Step 6: Result Handling (For Standard Operations)

For standard FHE operations (not decryption):

```
1. **Update ciphertext registry** with the new encrypted result !
[Bullet](../../../../static/img/assets/7.png)
```

At this point **\*\*the operation cycle is completed\*\***, preserving the confidentiality of all encrypted values

---

---

```
title: Decrypt/SealOutput from Cofhejs
sidebar_position: 4
```

— — —

```
Decrypt/SealOutput from Cofhejs
```

## ## Overview

This document lays out the complete flow of an off-chain sealoutput request. It is recommended to always use ``cofhejs.unseal`` rather than ``cofhejs.decrypt`` as ``unseal`` internally seals the user's data before returning it from the Threshold Network, making it inherently more secure (eg. man in the middle attack).

> Note: The example below is of ``cofhejs.unseal``, however ``cofhejs.decrypt`` uses the same API and returns the same result.

## ## Key Components

Component	Description
-----	
-----	
<b>**CtHash**</b>	The encrypted hash representing an encrypted number. Fetched on-chain.
<b>**cofhejs**</b>	Javascript library handling `permits` and the `unseal` / `decrypt` operations.
<b>**Threshold Network**</b>	Decentralized decryption network that handles the requests
<b>**ACL**</b>	On-chain <b>**A**</b> ccess <b>**C**</b> ontrol <b>**L**</b> ist responsible for tracking <b>**CtHash**</b> access.

— — —

## ## Flow Diagram

The following diagram illustrates the complete flow of an Decrypt/SealOutput request in the CoFHE ecosystem:

[[Diagram]](../../../../../static/img/assets/

```
offChain_sealoutput_decrypt.svg)](../../../../../static/img/assets/
offChain_sealoutput_decrypt.svg)
```

## ## Step-by-Step Flow

### ### 📌 Step 1: Fetching of CtHash

Solidity contract:

```
```solidity
contract

function setNumber(uint32 num) public {
    counter[msg.sender] = FHE.asEuint32(num);
    FHE.allowSender(counter[msg.sender]);
}

function getNumber() public view returns (euint64) {
    return counter[msg.sender];
}
```
```

1. Fetch the user's `euint64` from the chain by calling `const CtHash = await example.getNumber()` which returns an `euint64` as a js bigint.

> Note: All euints, along with ebool and eaddress, are wrappers around uint256. The data returned from `example.getNumber()` is in the type bigint, and can be treated as a `CtHash` directly

### ### 📌 Step 2: Integration with CoFHE.js

1. The decentralized application (dApp) integrates with CoFHE by utilizing `**Cofhejs**` for encryption.  
[See in GitHub](<https://github.com/FhenixProtocol/cofhejs>)
2. [Create a permit](../../cofhejs/permits-management.md) using `cofhejs.createPermit(...)` . This permit will automatically be used in the following step.
3. [Unseal](../sealing-unsealing.md) using `cofhejs.unseal(CtHash)` . Calls `/sealoutput` on the threshold network, unseals the result.

### ### 📌 Step 3 (Handled by cofhejs.unseal): <https://\{ThresholdNetworkUrl\}/sealoutput>

1. `**cofhejs.unseal` calls /sealoutput**`. The user's Permit is added to the request. `Permit.issuer` should be the `msg.sender` in Step 1 for the permit to be valid.

2. **Threshold Network** makes an on-chain call to the ``ACL`` to verify that the Permit is valid.

3. **ACL** verifies that the Permit is valid.

4. **ACL** verifies that ``Permit.issuer`` has been granted access to ``CtHash``. (Access is granted by ``FHE.allowSender`` in the Example contract function ``setNumber()``)

5. **Threshold Network** seals the data with ``Permit.sealingKey``

6. **Threshold Network** returns the sealed result to ``cofhejs``

### ### 📌 Step 4: Handling Results

``cofhejs`` receives the result from the Threshold Network and:

1. **Unseals the result** using the `private_key` of the sealing key pair. The result is always unsealed as a `bigint` regardless of the type of `CtHash` (`euint32` / `ebool` / `eaddress`)

2. **cofhejs** converts the output type as follows:

```
```typescript
export const convertViaUtype = <U extends FheTypes>(  
  utype: U,  
  value: bigint  
) : UnsealedItem<U> => {  
  if (utype === FheTypes.Bool) {  
    return !!value as UnsealedItem<U>;  
  } else if (utype === FheTypes.Uint160) {  
    return uint160ToAddress(value) as UnsealedItem<U>;  
  } else if (utype == null || FheUintUTypes.includes(utype as number))  
  {  
    return value as UnsealedItem<U>;  
  } else {  
    throw new Error(`convertViaUtype :: invalid utype :: ${utype}`);  
  }  
};
```

3. **The result is returned as a ``Result`` type**. The ``Result<T>`` type looks like this:

```
```typescript
export type Result<T, E = string> =
 | { success: true; data: T; error: null }
 | { success: false; data: null; error: E };
```

The `Result` type is a discriminated union that represents either:

- A successful operation with data (`success: true`)
  - A failed operation with an error message (`success: false`).
- The return type of `cofhejs.unseal` is determined by the utype passed in as the second argument:

```
```typescript
const boolResult: Result<bool> = await cofhejs.unseal(
  boolCtHash,
  FheTypes.Bool
);
const uintResult: Result<bigint> = await cofhejs.unseal(
  uintCtHash,
  FheTypes.Uint32
);
const addressResult: Result<string> = await cofhejs.unseal(
  ctHash,
  FheTypes.Address
);
```
```

```

title: CTRegistry
sidebar_position: 5

```

```
CTRegistry
Aspect	Description
Type	Registry Contract
Function	Manages the mapping between temporary ciphertext
hashes and their actual hash values	
Responsibilities	• Maintains a consistent record of ciphertext
identifiers throughout the CoFHE lifecycle • Enables secure lookup	
of final ciphertexts using their temporary handles • Restricts	
read/write access to ensure integrity and prevent unauthorized updates	
```

The CTRegistry acts as a source of truth for encrypted data identifiers, mapping temporary hashes to their final computed values. This ensures results from off-chain computation can be securely resolved and verified by their originating requests.

```

title: TaskManager
sidebar_position: 1

```

```
TaskManager
```

| Aspect                  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Type</b>             | Contract deployed on the destination blockchain                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <b>Function</b>         | Acts as the on-chain entry point for CoFHE integration                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <b>Responsibilities</b> | <ul style="list-style-type: none"> <li>Initiates FHE operations by serving as the on-chain entry point. The dApp contract calls the FHE.sol library which triggers the TaskManager contract to submit a new encrypted computation task.</li> <li>Generates unique handles that act as references to the results of FHE operations. These results are computed asynchronously off-chain.</li> <li>Emits structured events containing the unique handle of the ciphertext, operation type, and other required metadata.</li> </ul> |
| <b>Deployment</b>       | A separate Task Manager Contract is deployed for each supported destination chain, enabling chain-specific integrations                                                                                                                                                                                                                                                                                                                                                                                                          |

```

title: Fhe0s - Server
sidebar_position: 7

```

#### # Fhe0s - Server

| Aspect                  | Description                                                                                                                                                                                                                                                                                              |
|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Type</b>             | Off-chain computational layer                                                                                                                                                                                                                                                                            |
| <b>Function</b>         | Executes FHE operations and manages encrypted computations                                                                                                                                                                                                                                               |
| <b>Responsibilities</b> | <ul style="list-style-type: none"> <li>Receives the request from the Aggregator</li> <li>Executes the FHE operations</li> <li>Calls aggregator when result is created</li> <li>Returns plaintext results when requested (i.e decrypt/seal output), preserving privacy throughout the pipeline</li> </ul> |

The FHE Operating System server manages the execution environment for FHE operations.

```

title: ACL (Access Control Layer)
sidebar_position: 2

```

#### # ACL (Access Control Layer)

| Aspect                  | Description                                                                                                           |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <b>Type</b>             | Contract deployed on the destination blockchain                                                                       |
| <b>Function</b>         | Manages and enforces access control for ciphertexts, ensuring only authorized contracts can reference or decrypt them |
| <b>Responsibilities</b> | An internal contract that is responsible for managing and verifying access for each and every ciphertext              |

---  
title: Plaintexts Storage  
sidebar\_position: 4  
---

### # Plaintexts Storage

| Aspect                      | Description                                                                                                                                                                                                                                 |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -----                       | -----                                                                                                                                                                                                                                       |
| <b>**Type**</b>             | Internal Smart Contract                                                                                                                                                                                                                     |
| <b>**Function**</b>         | Storage and management of plaintext values in the host chain                                                                                                                                                                                |
| <b>**Responsibilities**</b> | • Manages the storage and retrieval of plaintext values in the system <br/> • Provides caching mechanism for plaintext values to improve retrieval performance <br/> • Ensures secure handling of decrypted data within the CoFHE ecosystem |

---  
title: Aggregator  
sidebar\_position: 3  
---

### # Aggregator

| Aspect                      | Description                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -----                       | -----                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>**Type**</b>             | Intermediary service between the blockchain and computation layer, ensuring structured request management.                                                                                                                                                                                                                                                                                                            |
| <b>**Function**</b>         | Manages and organizes the events (requests) received from the Task Manager Contract to ensure efficient processing.                                                                                                                                                                                                                                                                                                   |
| <b>**Responsibilities**</b> | • Listens to the emitted events on the destination chain<br/>• Processes incoming requests in FIFO (First In, First Out) to ensure fairness and consistency<br/>• Prepares and structures the data/events and submits the request to the fheOS server<br/>• Receives the results from the fheOS server and sends it to the Data Availability layer<br/>• Upon decryption result, submits the result to the host chain |

---  
title: Threshold Network  
sidebar\_position: 8  
---

### # Threshold Network

| Aspect          | Description                    |
|-----------------|--------------------------------|
| -----           | -----                          |
| <b>**Type**</b> | Offchain, distributed network. |

```
|
| **Function** | Process and execute decryption requests.
|
| **Responsibilities** | • Gets ciphertext decryption requests
•
Authenticates and validates them
• Runs an MPC protocol to decrypt
the ciphertext |
```

In any system utilizing encryption, a crucial step is the eventual decryption of data. For example, if we were to build a privacy-preserving ERC20 contract, users would ultimately need to access their encrypted balances. In the case of CoFHE, this decryption process is managed by the Threshold Network.

## ## Motivation

The Threshold Network is a component of a complex cryptographic system with the sole purpose of enhancing the security and trustworthiness of the system by distributing control of the decryption process. Rather than having a single secret key stored and used for the decryption by a centralized entity, we distribute secret shares (i.e. shares of a decryption key) among multiple parties. This enforces collaboration among parties in order to decrypt; the parties perform an MPC (Multi-Party Computation) protocol that results in the decrypted value of a given ciphertext, ensuring that no information about the full secret key is leaked at any time.

A practical example of a threshold network in practice is vote counting. Multiple representatives of competing parties gather around to count votes from recent elections. In order to attempt voter fraud all of the participating parties would have to collaborate (which is unlikely). Threshold Network is built on the exact same principle.

## ## Concept

Threshold Network performs decryption operations. The Threshold Network is currently initialized by a Trusted Dealer (in the future, we plan to eliminate the Trusted Dealer). The Dealer initially generates a key. Then, the Dealer uses a secret-sharing algorithm to generate Secret Shares of the key to share among individual members. Each member holds exactly one secret share. To perform a decryption, the secret shares are used to perform partial decryptions through a multiparty computation (MPC) protocol. These partial decryptions are then combined into the final plaintext. The protocol requires cooperation from all participants to perform a decryption, ensuring no single entity can decrypt the ciphertext alone. This distributed control mechanism enhances security by preventing unilateral access to encrypted data.

## ## Decryption Process

The Threshold network includes three main components:

- Coordinator – coordinates communication between the party members to perform the MPC protocol.
- Party Members – the individual parties that hold a secret share and execute the MPC protocol.
- Trusted Dealer – responsible for initializing the protocol, and for providing random data to the party members, needed to perform the protocol securely.

![Threshold Network Flow](../../static/img/threshold-network.svg)

All incoming decryption requests reach the Coordinator (1). The coordinator splits the CT into individual Learning With Errors (LWE) CT blocks. These blocks then get broadcast to partymembers (2). During this process data is exchanged back and forth until the decryption of all blocks is complete upon which the coordinator reassembles the plaintext from decrypted LWE CT blocks. The plaintext value then gets sent back to the user.

The MPC protocol consists of multiple stages. In each stage, a partymember performs a calculation on a received input and returns the result (a.k.a. intermediate result) to the Coordinator. Each intermediate result gets sent back to the coordinator in order to get distributed among other partymembers as an input for the next stage.

---  
title: ZK Verifier  
sidebar\_position: 6  
---

## # ZK Verifier

**\*\*ZK-Verifier\*\*** is an essential component for encrypting and providing data as inputs to confidential smart contracts.

| Aspect                      | Description                                                                                                                                                               |
|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -----                       |                                                                                                                                                                           |
| -----                       |                                                                                                                                                                           |
| -----                       |                                                                                                                                                                           |
| <b>**Type**</b>             | Off-chain service, used by clients.                                                                                                                                       |
| <b>**Function**</b>         | Verifies the user's input, ensuring that it is safe to use.                                                                                                               |
| <b>**Responsibilities**</b> | • Receives a user's ZKPoK of their inputs.<br/>• Verifies said ZK proofs.<br/>• Generates a signature, allowing the user to use these inputs in a smart contract function |

call. |

## ## ZKPoK – Why?

**\*\*Zero-Knowledge Proof of Knowledge (ZKPoK)\*\*** provides a crucial security mechanism in CoFHE. It allows users to prove they know the plaintext of an encrypted input they're sending to a smart contract, without revealing the plaintext itself.

ZKPoKs protect against potential malicious vectors, including:

1. **Malleability Attacks:** Without ZKPoK protection, attackers could manipulate encrypted data by applying transformations to observed ciphertexts, even without knowing what's inside them. For example, they might combine existing ciphertexts with encrypted zero values to create new valid-looking encrypted data, potentially compromising user's confidentiality.
2. **Chosen Ciphertext Attacks (CCAs):** Attackers can submit modified ciphertexts to the system and observe the results, potentially exploiting homomorphic operations to infer sensitive information, manipulate data, or even recover the secret key.

Requiring a ZKPoK for each encrypted input helps running an encryption system in the public space that is a blockchain runtime. It ensures that only users with knowledge of the original plaintext can produce valid proofs. This approach eliminates multiple security risks, protecting sensitive user data and maintaining the system's integrity.

## ## Sending encrypted inputs

As mentioned before, when providing ciphertexts as an input to a smart contract, users have to generate a ZKPoK and get a verification approval first. Although most of the work will be handled by `cofhe.js` and `FHE.sol`, we will describe this mechanism in high-level (also in the diagram below).

![ZK Proof of Knowledge Flow](../../static/img/zk-pok.svg)

The process of sending input(s) to a Smart Contract:

1. User encrypts an input(s) and generate a ZK proof of knowledge for it.
2. User sends the ciphertext(s) and proof(s) to the ZKVerifier.
3. ZK-Verifier verifies the proof. If valid, sign a message that approves input(s).
4. ZK-Verifier returns the signed approval to User.
5. User sends `(ciphertext, signed\_approve)` (one or more) as input(s) to a contract call.
6. Contract verifies the signed message, approving the input(s).
7. Contract performs actual logic.

As mentioned, most of this process is abstracted away. In fact, steps 1-6 are all handled behind the scenes, while step 7 (the actual logic) is, of course, up to the user to write.

## ## ZK-Verifier

The ZK-Verifier is a zk-verification program. It has two purposes:

1. Verify the ZKPoK's of ciphertexts that are intended to be inputted to a CoFHE smart contract.
2. Sign a verification message, allowing the said contract to ensure that the inputs are safe and were validated.

The signed message that the ZK-Verifier outputs will then be verified by the receiving contract using `ecrecover`. That means that the ZKVerifier's public key will be predetermined and well-known.

The ZKVerifier is intended to run in a TEE to reduce trust and ensure the integrity of the inputs and signed verification messages.

---

title: CoFHE.js  
sidebar\_position: 1

---

```
import Tabs from "@theme/Tabs";
import TabItem from "@theme/TabItem";
```

### # CoFHE.js

| Aspect                      | Description                                                                                                          |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------|
| -----                       |                                                                                                                      |
| -----                       |                                                                                                                      |
| <b>**Type**</b>             | Typescript Library                                                                                                   |
| <b>**Function**</b>         | Provides functionality to initialize TFHE, generate and manage permits, encrypt input data, and read encrypted data. |
| <b>**Responsibilities**</b> | Handle primary interactions with FHE enabled contracts and the CoFHE Co-Processor.                                   |
|                             |                                                                                                                      |

CoFHE.js is the JavaScript library that provides client-side functionality for interacting with CoFHE smart contracts. It handles encryption, decryption, and communication with the blockchain.

## ## Installation

To get started with cofhejs, you need to install it as a dependency in your JavaScript project. You can do this using npm (Node Package Manager) or Yarn. Open your terminal and navigate to your project's directory, then run the following:

```
<Tabs>
 <TabItem value="yarn" label="yarn">
    ``` yarn add cofhejs ```
  </TabItem>
  <TabItem value="npm" label="npm">
    ``` npm install cofhejs ```
 </TabItem>
 <TabItem value="pnpm" label="pnpm">
    ``` pnpm add cofhejs ```
  </TabItem>
</Tabs>
```

> Note: _For more information on getting started, take a look at the [\[**cofhejs getting started**\]\(/docs/devdocs/quick-start/getting-started\)](/docs/devdocs/quick-start/getting-started) guide._

Key Features

Initialization

```
```javascript
const { cofhejs } = require('cofhejs/node')
const { ethers } = require('ethers')

// initialize your web3 provider
const provider = new ethers.JsonRpcProvider('http://127.0.0.1:42069')
const wallet = new ethers.Wallet(PRIVATE_KEY, provider)

// initialize cofhejs Client with ethers (it also supports viem)
await cofhejs.initializeWithEthers({
 ethersProvider: provider,
 ethersSigner: wallet,
 environment: 'LOCAL',
})
```
```

> Note: _For more information on getting started, take a look at the [\[**cofhejs getting started**\]\(/docs/devdocs/quick-start/getting-started\)](/docs/devdocs/quick-start/getting-started) guide._

- Client-side encryption/decryption

```
```javascript
const logState = (state: EncryptStep) => {
 console.log(`Log Encrypt State :: ${state}`);
}
```

```
};

// This will encrypt only the encrypted values (total 4 in this case)
const encryptedValues = await cofhejs.encrypt(logState, [
 { a: Encryptable.bool(false), b: Encryptable.uint64(10n), c:
 "hello" },
 ["hello", 20n, Encryptable.address(contractAddress)],
 Encryptable.uint8("10"),
] as const);

const contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI,
wallet);
// Use the encrypted value of 10n
const tx = await contract.add(encryptedValues.data[1]);
```

```

Cofhejs exposes a function ``encrypt`` that takes any data type, any ``Encryptable`` values within the input are extracted into an array, packed, and verified using a ZK Proof of Encryption. For more information on this process and how it works under the hood take a look at the [encryption data flow](../data-flows/encryption-request.md).

– Key management

Cofhejs automatically fetches two keys from CoFHE, the FHE public key and a CRS. These keys are used during the ``encrypt`` flow, which prepares encrypted inputs to be used as transaction parameters.

> Note: [_Read more about encryption \[here\]\(/docs/devdocs/cofhejs/encryption-operations\)_](#)

– Permits

Permits allow users to access their encrypted data by authenticating the user with an EIP712 signature. Permits can be managed within cofhejs, or they can be managed within a client dApp directly by using the exposed ``Permit`` class.

> Note: [_Read more about permits \[here\]\(/docs/devdocs/cofhejs/permits-management\)_](#)

– Reading encrypted data

On-chain FHE operations are symbolic, meaning that an ``euint64`` is a handle to an encrypted number that lives on the Fhenix Mainnet chain. It is possible to read the encrypted data through an off-chain call by using ``cofhejs.unseal()`` like so:

```
```typescript
```

```
const resultHandle = await contract.getSomeEncryptedUint32()
const unsealed = await cofhejs.unseal(resultHandle, FheTypes.Uint32)
```
```

You can read more about unsealing and the underlying process in the [unsealing data-flow page](../data-flows/decrypt-seal-output.md), and you can read more about `cofhejs.unseal` usage [here](/docs/devdocs/cofhejs/sealing-unsealing).

```
---
title: FHE.sol
sidebar_position: 2
---
```

FHE.sol

| Aspect | Description |
|-------------------------|---|
| Type | Solidity library |
| Function | Provides a user-facing implementation of all Fully Homomorphic Encryption (FHE) operations, supporting every compatible input type. |
| Responsibilities | Implements all FHE operations, making them accessible within Solidity smart contracts. |

Usage

```
```solidity
import {FHE} from "@fhenixprotocol/cofhe-contracts/FHE.sol";
```
```

For a complete reference of all available functions and their usage, check out the [FHE.sol API documentation](../solidity-api/FHE.md).

```
---
title: Error Handling
sidebar_position: 7
---
```

Error Handling

`cofhejs` uses a consistent error handling pattern based on the `Result` type to provide predictable and type-safe error handling throughout the library. This guide explains how error handling works and how to properly handle errors in your applications.

The Result Type

`cofhejs` uses a functional approach to error handling with the `Result` type. This pattern avoids exceptions and provides explicit error information.

```

```typescript
export type Result<T, E = string> = { success: true; data: T; error:
null } | { success: false; data: null; error: E }
```

```

The `Result` type is a discriminated union that represents either:

- A successful operation with data (`success: true`)
- A failed operation with an error message (`success: false`)

Helper Functions

`cofhejs` provides two helper functions to create Result objects:

```

```typescript
// Creates a Result representing a failure
export const ResultErr = <T, E>(error: E): Result<T, E> => ({
 success: false,
 data: null,
 error,
})

// Creates a Result representing a success
export const ResultOk = <T, E>(data: T): Result<T, E> => ({
 success: true,
 data,
 error: null,
})
```

```

Where Result is Used

Most asynchronous operations in `cofhejs` return a `Result` type, including:

- Initialization functions (`initializeWithEthers`, `initializeWithViem`, `initialize`)
- Permit operations (`createPermit`, `getPermit`, `getPermission`)
- Encryption and decryption operations

Handling Errors

When working with functions that return a `Result`, always check the `success` property before accessing the data.

Basic Error Handling Pattern

```

```typescript
const result = await cofhejs.initialize({

```

```

 provider: ethersProvider,
 signer: wallet,
 environment: 'TESTNET',
 })

 if (!result.success) {
 console.error('Initialization failed:', result.error)
 // Handle the error appropriately
 return
 }

 // Safe to access result.data only after checking success
 const permit = result.data
 // Continue with your application logic
 ``

```

### ### Error Handling with Destructuring

You can use destructuring to make your code more concise:

```

````typescript
const {
    success,
    data: permit,
    error,
} = await cofhejs.createPermit({
    type: 'self',
    issuer: userAddress,
})

if (!success) {
    console.error('Failed to create permit:', error)
    return
}

// Use permit safely
console.log('Permit created successfully:', permit)
````

```

### ## Common Error Scenarios

`cofhejs` may return errors in various scenarios, including:

#### 1. **\*\*Initialization Errors\*\***:

- Missing provider or signer
- Network connectivity issues
- Unsupported environment

#### 2. **\*\*Permit Errors\*\***:

- Invalid permit parameters
- Missing signer
- Unauthorized operations

### 3. **\*\*Encryption Errors\*\***:

- Missing FHE public key
- Invalid input types
- Network service unavailability

### ## Complete Example

Here's a complete example of initializing `cofhejs` and handling potential errors:

```
```typescript
async function initializeCoFHE() {
    try {
        // initialize your web3 provider
        const provider = new
ethers.BrowserProvider(window.ethereum)
        const signer = (await provider.getSigner()) as
ethers.JsonRpcSigner

        // initialize cofhejs Client with ethers (it also
supports viem)
        await cofhejs.initializeWithEthers({
            provider: window.ethereum,
            signer: wallet,
            environment: 'TESTNET',
        })

        if (!result.success) {
            // Handle specific error cases
            if (result.error.includes('missing provider'))
{
                console.error('Provider not available.
Please install a wallet extension.')
            } else if (result.error.includes('failed to
initialize cofhejs')) {
                console.error('FHE initialization
failed. The network may not be FHE-enabled.')
            } else {
                console.error('Initialization error:',
result.error)
            }
            return null
        }

        console.log('`cofhejs` initialized successfully')
```

```

        return result.data // The permit, if generated
    } catch (unexpectedError) {
        // Catch any unexpected errors not handled by the
Result pattern
        console.error('Unexpected error during
initialization:', unexpectedError)
        return null
    }
}

// Example of creating and using a permit with error handling
async function createAndUsePermit(userAddress) {
    const permitResult = await cofhejs.createPermit({
        type: 'self',
        issuer: userAddress,
    })

    if (!permitResult.success) {
        console.error('Permit creation failed:',
permitResult.error)
        return
    }

    const permit = permitResult.data
    console.log('Permit created successfully:', permit)

    // Continue with operations that require the permit
    // ...
}

```

Testing Error Cases

When writing tests, `cofhejs` provides utility functions to validate error results:

```

```typescript
import { expectResultError } from 'cofhejs/test'

test('should return error for invalid parameters', async () => {
 const result = await cofhejs.initialize({
 // Missing required parameters
 })

 expectResultError(result, 'initialize :: missing provider -
Please provide an AbstractProvider interface')
})

```

By consistently checking the `success` property and appropriately

handling errors, you can build robust applications that gracefully handle failure cases when working with `cofhejs`.

---

sidebar\_position: 2

---

## # Encryption

cofhejs provides an easy-to-use function to encrypt your inputs before sending them to the Fhenix Co-Processor.

:::tip

Encryption in Fhenix is done using the global chain key. This key is loaded when you create a cofhejs client automatically

:::

When we perform encryption, we specify the type of `euint` (Encrypted Integer) we want to create. This should match the expected type in the Solidity contract we are working with.

First, initialize the library:

```
```Typescript
await cofhejs.initializeWithEthers({
  ethersProvider: provider,
  ethersSigner: wallet,
  environment: "LOCAL",
});
```

// or

```
await cofhejs.initializeWithViem({
  viemClient: provider,
  viemWalletClient: wallet,
  environment: "LOCAL",
});
```

...

Then, you can use the created client to encrypt

```
```Typescript
import { cofhejs, Encryptable } from "cofhejs/node";
```

```
// Initialize cofhejs
```

```
const logState = (state) => {
 console.log(`Log Encrypt State :: ${state}`);
};
```

```
let result: EncryptableBool = await cofhejs.encrypt(logState,
```

```

[EncryptionTypes.bool(true)]];
let result: EncryptableUint8 = await cofhejs.encrypt(logState,
[EncryptionTypes.uint8(10)]);
let result: EncryptableUint16 = await cofhejs.encrypt(logState,
[EncryptionTypes.uint16(10)]);
let result: EncryptableUint32 = await cofhejs.encrypt(logState,
[EncryptionTypes.uint32(10)]);
let result: EncryptableUint64 = await cofhejs.encrypt(logState,
[EncryptionTypes.uint64(10)]);
let result: EncryptableUint128 = await cofhejs.encrypt(logState,
[EncryptionTypes.uint128(10)]);
let result: EncryptableUint256 = await cofhejs.encrypt(logState,
[EncryptionTypes.uint256(10)]);
let result: EncryptableAddress = await cofhejs.encrypt(logState,
[EncryptionTypes.address("0x1234567890123456789012345678901234567890")
]);
...

```

Or, we can use the nested form to encrypt multiple values at once:

```

```javascript

let result = await cofhejs.encrypt(logState,
[EncryptionTypes.bool(true), EncryptionTypes.uint8(10),
EncryptionTypes.uint16(10), EncryptionTypes.uint32(10),
EncryptionTypes.uint64(10), EncryptionTypes.uint128(10),
EncryptionTypes.uint256(10),
EncryptionTypes.address("0x1234567890123456789012345678901234567890")])
);
...

```

The returned types from the encrypt function will be of the type ``CoFheInUint8``, ``CoFheInUint16`` or ``CoFheInUint32`` (or 64/128/256 etc.) depending on the type you specified.

The ``EncryptedUint`` types sound scary, but are actually pretty simple. It's just a

```

```typescript
export interface EncryptedNumber {
 data: Uint8Array;
 securityZone: number; // defaults to 0
}

export interface EncryptedUint8 extends EncryptedNumber {}
...

```

These types exist in order to enable type checking when interacting

with Solidity contracts, and to make it easier to work with encrypted data.  
However, feel free to use the `data` field directly if you prefer.

### ### setState

The setState function is used to monitor the state of the encryption process.

Since the process is asynchronous, we can use this function to get the state of the encryption process for better UI experience.

```
```typescript
const logState = (state) => {
  console.log(`Log Encrypt State :: ${state}`);
};
```
```

The available states are:

- Extract - Getting all the data ready for encryption (values to encrypt, chain information, etc.)
- Pack - Preparing the data for the encryption process
- Prove - Signing the data
- Verify - Verifies the user's input, ensuring that it is safe to use (read more about this [\[here\]](/docs/devdocs/architecture/internal-utilities/verifier))
- Replace - Preparing the result and replacing the old values with encrypted ones
- Done - Process is finished

---

title: Permits Management  
sidebar\_position: 4

---

## # Permits Management

Permits are a crucial security mechanism in Fhenix that allow users to authenticate themselves when accessing encrypted data through off-chain operations like `sealoutput` and `decrypt`. These operations are exposed and handled by `cofhejs`.

## ## Quick Start

### ### Basic Integration

In a development environment, permit management can be handled automatically by `cofhejs`. When initialized with a valid provider and signer, the SDK will prompt users to sign a new Permit, granting them access to their encrypted data:

```

```typescript
// initialize your web3 provider
const provider = new ethers.JsonRpcProvider('http://127.0.0.1:42069')
const wallet = new ethers.Wallet(PRIVATE_KEY, provider)

// initialize cofhejs Client with ethers (it also supports viem)
await cofhejs.initializeWithEthers({
  ethersProvider: provider,
  ethersSigner: wallet,
  environment: 'LOCAL',
})
```

```

### ### Production Setup

For production environments, you'll want more control over the permit generation process. Disable automatic permit generation by setting `generatePermit: false`:

```

```typescript
// initialize your web3 provider
const provider = new ethers.JsonRpcProvider('http://127.0.0.1:42069')
const wallet = new ethers.Wallet(PRIVATE_KEY, provider)

// initialize cofhejs Client with ethers (it also supports viem)
await cofhejs.initializeWithEthers({
  provider: provider,
  signer: signer,
  environment: 'MAINNET',
  generatePermit: false,
})
```

```

After initialization, you'll need to manually call `cofhe.createPermit()` to generate user permits. It's recommended to inform users about the purpose of permits before requesting their signature.

### ### User Interface Example

Here's an example modal that explains permits to users:

```

```jsx
const PermitModal = () => (
  <div className='permit-modal'>
    <h2>Sign a Permit</h2>
    <p>Permits grant secure access to your encrypted data
on Fhenix by authenticating you with your signature. Each permit:</p>
    <ul>

```

```

        <li>Is valid for 24 hours</li>
        <li>Can only be used by you</li>
        <li>Ensures your data remains private</li>
      </ul>
      <button onClick={handleSignPermit}>Sign Permit</
button>
    </div>
  )
  ...

```

The sign permit action should call:

```

```typescript
const handleSignPermit = async () => {
 const result = await cofhejs.createPermit({
 type: 'self',
 issuer: userAddress,
 })

 if (!result.success) {
 console.error('Failed to create permit:',
result.error)
 return
 }
 // Permit created successfully
}
...

```

This will trigger the user's wallet to prompt for their signature. Once signed, the permit will be automatically stored and used for subsequent `cofhe.unseal` operations.

## ## Sharing Permits

### ### Overview

Sharing permits enables users to grant others access to their encrypted data in a secure way. The sharing process involves three steps:

1. Original data owner creates a sharing permit
2. Owner sends the permit to the recipient
3. Recipient activates the permit with their signature

### ### Implementation

Here's how to implement permit sharing:

1. Data owner creates a sharing permit:

```

```typescript
const createSharingPermit = async (ownerAddress: string,
recipientAddress: string) => {
    const result = await cofhejs.createPermit({
        type: 'sharing',
        issuer: ownerAddress,
        recipient: recipientAddress,
    })

    if (!result.success) {
        throw new Error(`Failed to create sharing permit: $
{result.error}`)
    }

    return result.data
}
```

```

2. The permit data can be safely transmitted to the recipient as plaintext since it contains no confidential information.

3. Recipient activates the permit:

```

```typescript
const activateReceivedPermit = async (sharingPermit: Permit) => {
    const result = await cofhejs.createPermit({
        ...sharingPermit,
        type: 'recipient',
    })

    if (!result.success) {
        throw new Error(`Failed to activate permit: $
{result.error}`)
    }

    return result.data
}
```

```

## ## Advanced Features

### ### Permit Validation

Permits include built-in validation mechanisms:

- **\*\*Expiration\*\***: Permits automatically expire after 24 hours (configurable)
- **\*\*Signature Verification\*\***: Ensures permits are only used by authorized parties
- **\*\*Chain Validation\*\***: Permits are bound to specific networks

### ### Custom Validators

You can implement custom validation logic by specifying a validator contract:

```
```typescript
const permitWithValidator = await cofhejs.createPermit({
  type: 'self',
  issuer: userAddress,
  validatorContract: '0x...', // Your validator contract address
  validatorId: 1, // Custom validation identifier
})
```
```

### ### Error Handling

Always handle permit operations with proper error checking:

```
```typescript
const handlePermitCreation = async () => {
  try {
    const result = await cofhejs.createPermit({
      type: 'self',
      issuer: userAddress,
    })

    if (!result.success) {
      console.error('Permit creation failed:',
result.error)
      return
    }

    // Handle successful permit creation
  } catch (error) {
    console.error('Unexpected error:', error)
  }
}
```
```

### ## API Reference

See `PermitOptions` interface for the complete list of options available when creating permits:

```
```typescript
interface PermitOptions {
  type: 'self' | 'sharing' | 'recipient'
  issuer: string
  recipient?: string
}
```

```

        expiration?: number
        validatorId?: number
        validatorContract?: string
        name?: string
    },
    ...

```

```

---
title: Sealing & Unsealing
sidebar_position: 3
---

```

Overview

In Fhenix's FHE system, data returned from smart contracts is "sealed" (internally re-encrypted since it already exists in an encrypted state) to maintain confidentiality during transmission. The unsealing process converts this encrypted data back into readable values using your permit's sealing key pair.

> Note: To learn more about sealed box encryption, take a look at the [libsodium sealedbox docs](https://libsodium.gitbook.io/doc/public-key_cryptography/sealed_boxes#purpose).

Basic Usage

Simple Unsealing

The most straightforward way to unseal data is using ``cofhejs.unseal()``:

> Note: Unsealing requires ``cofhejs`` to be [initialized](/docs/devdocs/quick-start/getting-started.md) and for a [permit](./permits-management.md) to be created.

```

```typescript
// Get sealed data from a contract
const sealedBalance = await myContract.getBalance()

// Unseal with the correct type
const result = await cofhejs.unseal(sealedBalance, FheTypes.Uint64)
if (!result.success) {
 console.error('Failed to unseal:', result.error)
 return
}

console.log('Balance:', result.data) // Unsealed value as BigInt
```

```

Supported Types

The unsealing process supports all FHE data types:

```
```typescript
// Integer types
const uint8 = await cofhejs.unseal(sealed, FheTypes.Uint8)
const uint16 = await cofhejs.unseal(sealed, FheTypes.Uint16)
const uint32 = await cofhejs.unseal(sealed, FheTypes.Uint32)
const uint64 = await cofhejs.unseal(sealed, FheTypes.Uint64)
const uint128 = await cofhejs.unseal(sealed, FheTypes.Uint128)
const uint256 = await cofhejs.unseal(sealed, FheTypes.Uint256)

// Boolean
const bool = await cofhejs.unseal(sealed, FheTypes.Bool)

// Address
const address = await cofhejs.unseal(sealed, FheTypes.Address)
```
```

Advanced Usage

Direct Permit Unsealing

For lower-level control, you can use the Permit class directly to unseal data:

```
```typescript
const permit = await Permit.create({
 type: 'self',
 issuer: userAddress,
})

// Seal some data (for demonstration)
const value = 937387n
const sealed = SealingKey.seal(value, permit.sealingPair.publicKey)

// Unseal directly with permit
const unsealed = permit.unseal(sealed)
console.log(unsealed === value) // true
```
```

Type Conversions

Internally, data types require specific handling when unsealed:

```
```typescript
// Boolean values
const boolValue = true
const sealedBool = SealingKey.seal(boolValue ? 1 : 0,
 permit.sealingPair.publicKey)
const unsealedBool = permit.unseal(sealedBool)
```

```

const resultBool = unsealedBool === 1n // Convert BigInt to boolean

// Address values
const addressValue = '0x1234...'
const sealedAddress = SealingKey.seal(BigInt(addressValue),
permit.sealingPair.publicKey)
const unsealedAddress = permit.unseal(sealedAddress)
const resultAddress = getAddress(`0x$
{unsealedAddress.toString(16).slice(-40)}`)
`

```

However this is handled for you with ``cofhejs.unseal``. Unsealing an encrypted boolean will return a ``bool``, an encrypted address will return a ``0x`` prefixed string, and an encrypted number will return a `js `bigint``.

```

title: Data Evaluation
sidebar_position: 2

```

## # Data Evaluation

### ## Sending computation requests

The blockchain that we write Smart Contracts on (for example, Arbitrum One) does not natively support FHE computation. This is why CoFHE is mostly an *off-chain system*, performing all the FHE heavy lifting asynchronously. All the logic that is happening on-chain is *giving instructions* for the off-chain component, CoFHE's *FHE Engine*, on what to compute. This concept is commonly referred to as [Symbolic Execution](https://en.wikipedia.org/wiki/Symbolic\_execution).

### #### But how the on-chain smart contracts communicate with the off-chain engine?

Through *Events*. Every FHE operation exposed in ``FHE.sol`` that requires an FHE computation emits an event. For example:

```

`sol
res = FHE.sub(first, second);
`

```

This code snippet above computes subtraction between two numbers. Behind the scenes, the function ``FHE.sub()`` is *emitting an event*, basically broadcasting "Hey FheOS! you need to compute ``first - second``!". CoFHE then picks up this event, and forwards it to FheOS (the compute engine) for execution.

:::note[Some more examples]

```
```sol
euint8 res = FHE.asEuInt8(42);
```
```

This command above emits an event saying "Create a trivially encrypted ciphertext representing the plaintext number `42`".

```
```sol
balance = FHE.add(amount, balance);
```
```

This command above emits an event saying "Compute the encrypted result of adding the encrypted variables `balance` and `amount`".

:::

But wait, how does CoFHE know to connect between the variables, e.g. `balance` and `amount`, and the actual underlying encrypted data to calculate the result? To understand this, we need to understand how encrypted data is represented in the smart contracts 📌.

## ## Data Representation

In the context of a Smart Contract, most FHE operation results in a new ciphertext. Let's look at an example:

```
```sol
function addNumbers() public view returns (euInt32) {
    euInt32 a = FHE.asEuInt32(10); // Creating two trivially-encrypted
    ciphertexts
    euInt32 b = FHE.asEuInt32(20);
    euInt32 result = FHE.add(a, b); // Add them together

    return result;
}
```
```

In the example above, we are:

1. Creating two trivially-encrypted 32-bit ciphertexts using `FHE.asEuInt32()`.
2. Perform an FHE-addition, calculating the encrypted sum of both, using `FHE.add()`.
3. Returning the result.

We can see that the result of every operation is a value of type `euInt32`, which represents a new 32-bit ciphertext. But what does `euInt32` represent exactly? Let's look at the type's declaration:

```
```sol
```

```
type uint32 is uint256;  
```
```

So is it actually a plaintext `uint256` integer 🤔? Well, not exactly.

The actual ciphertext value of FHE-encrypted integers is too big to be stored directly in the blockchain, or emitted in an event. That's why in your smart contracts, the ciphertext are being represented by a 256-bit handle regardless of their encrypted type. You can think of this as an ID, or a pointer. This handle is the identifier of said ciphertext. In practice, CoFHE actually stores full ciphertexts in an off-chain Data Availability (DA) layer.

So, when evaluating the following statement:

```
```sol  
ebool isBigger = FHE.gt(newBid, currentBid);  
```
```

FHE.sol is actually emitting the following event: "Check which number is bigger: `0xab12...` or `0xcd34...`". The result's handle (or, identifier) will be stored the variable `isBigger`, of type `ebool`.

:::tip

What to do with this `ebool isBigger`? Check out the page on [\[conditions\]\(select-vs-ifelse.md\)](#).

:::

:::tip[Deep Dive]

Didn't we just said that the computation is executed asynchronously? So – how can we know the ciphertext's handle in real time?

In fact, the ciphertext's handle is determined regardless of it's value. It is basically representing the operation that needs to be performed to create this value.

For example:

```
```sol  
uint64 num = FHE.asEuint64(31);  
uint64 meaning = FHE.add(num, FHE.asEuint64(11));  
```
```

The handle of `num` is a numerical representation of "trivially-encrypted `31`", while the handle of `meaning` is a similar representation of "result of addition between `num` and trivially-encrypted `11`". The actual encrypted value is, as mentioned before,

evaluated asynchronously.

```
:::
```

```

```

```
title: Access Control
```

```
sidebar_position: 5
```

```

```

```
Access Control
```

```
Motivation
```

Consider the following scenario: Your contract receives an encrypted input which should be kept confidential.

```
```solidity
// Contract A
function submitSecretBid(InEuint32 bid) {
    euint32 handle = FHE.asEuint32(bid);
    // Perform some operations on the encrypted input
}
```
```

If there were no access control mechanisms, someone could inspect the handle value which was received above, and then initialize a local variable with the same handle value as the one received above.

```
```solidity
// Contract B
function attackBid(uint256 seenHandleValue) {
    FHE.decrypt(seenHandleValue); // try to expose the value
}
```
```

Therefore, all operations check that the issuer of the operation has permission to use the ciphertext handle.

So in practice, the code above will raise a `ACLNotAllowed` error when trying to decrypt the ciphertext.

Any other operation will also fail if the issuing contract does not have allowance on all the inputs, such as:

```
```solidity
FHE.add(notAllowedCt, allowedCt); // -> ACLNotAllowed
```
```

Every newly created ciphertext handle is allowed to the creating contract for the duration of the transaction, and any other allowance must be explicitly granted by this contract, with the following Solidity API:

1. `FHE.allowThis(CIPHERTEXT\_HANDLE)` – allows the current contract

access to the handle.

2. ``FHE.allow(CIPHERTEXT_HANDLE, ADDRESS)`` - allows the specified address access to the handle.

3. ``FHE.allowTransient(CIPHERTEXT_HANDLE, ADDRESS)`` - allows the specified address access to the handle for the duration of the transaction.

:::note[Note]

For a user to be able to decrypt a ciphertext off-chain via the decryption network, the issuer must be allowed on the ciphertext handle.

:::

See [ACL Usage Examples](../tutorials/acl-usage-examples) for examples on how to explicitly manage ciphertext allowances in contracts.

### Behind the scenes

Every blockchain integrating CoFHE will deploy a contract called ``ACL.sol``.

This contract manages ownership records for each ciphertext, ensuring that only authorized owners can perform operations on their encrypted data.

The ACL contract contains the following mapping which tracks the ownership of each ciphertext handle:

```
```solidity
```

```
mapping(uint256 handle => mapping(address account => bool isAllowed))
persistedAllowedPairs;
```

```
```
```

```

```

```
title: FHE Encrypted Operations
```

```
sidebar_position: 3
```

```

```

# FHE Encrypted Operations

The library exposes utility functions for FHE operations. The goal of the library is to provide a seamless developer experience for writing smart contracts that can operate on confidential data.

## Types

The library provides a type system that is checked both at compile time and at run time. The structure and operations related to these types are described in this sections.

We currently support encrypted integers of bit length up to 256 bits and special types such as ``ebool`` and ``eaddress``.

The encrypted integers behave as much as possible as Solidity's integer types. However, behavior such as "revert on overflow" is not

supported as this would leak some information of the encrypted integers. Therefore, arithmetic on `euint` types is [unchecked] (<https://docs.soliditylang.org/en/latest/control-structures.html#checked-or-unchecked-arithmetic>), i.e. there is wrap-around on overflow.

In the back-end, encrypted integers are FHE ciphertexts. The library abstracts away the ciphertexts and presents pointers to ciphertexts, or ciphertext handles, to the smart contract developer. The `euint`, `ebool` and `eaddress` types are \_wrappers\_ over these handles.

```
<table>
<tr><th colspan="2"> Supported types </th></tr>
<tr><td>
```

name	Bit Size	Usage
`euint8`	8	Compute
`euint16`	16	Compute
`euint32`	32	Compute
`euint64`	64	Compute
`euint128`	128	Compute
`euint256`	256	Compute
`ebool`	8	Compute
`eaddress`	160	Compute

```
</td></td>
```

name	Bit Size	Usage
`InEuint8`	8	Input
`InEuint16`	16	Input
`InEuint32`	32	Input
`InEuint64`	64	Input
`InEuint128`	128	Input
`InEuint256`	256	Input
`InEbool`	8	Input
`InEaddress`	160	Input

```
</td></tr> </table>
```

## ## Operations

There are two ways to perform operations with FHE.sol:

### ### Using Direct Function Calls

Direct function calls are the most straightforward way to perform operations with FHE.sol. For example, if you want to add two encrypted 8-bit integers (euint8), you can do so as follows:

```
```javascript
euint8 result = FHE.add(lhs, rhs);
```

```

Here, lhs and rhs are your uint8 variables, and result will store the outcome of the addition.

### ### Using Library Bindings

FHE.sol also provides library bindings, allowing for a more natural syntax. To use this, you first need to include the library for your specific data type. For uint8, the usage would look like this:

```
```javascript
uint8 result = lhs.add(rhs);
```
```

In this example, lhs.add(rhs) performs the addition, using the library function implicitly.

```
:::tip
The `ebool` type is not a real boolean type. It is implemented as a
`uint8`
:::
```

### ## Supported Operations

```
:::tip
A documentation of every function in FHE.sol (including inputs and
outputs) can be found in [FHE.sol](../solidity-api/FHE.md)
:::
```

All operations supported by FHE.sol are listed in the table below.

Note that all functions are supported in both direct function calls and library bindings.

| Name                   |                        | FHE.sol function       |                        | Operator               | uint8                  |
|------------------------|------------------------|------------------------|------------------------|------------------------|------------------------|
| uint16                 | uint32                 | uint64                 | uint128                | uint256                | ebool                  |
| eaddress               |                        |                        |                        |                        |                        |
| -----                  |                        | -----                  | -----                  | -----                  | -----                  |
| -----                  | -----                  | -----                  | -----                  | -----                  | -----                  |
| -: :-----:             |                        |                        |                        |                        |                        |
| Addition               |                        | `add`                  |                        | `+`                    | <g>&gt;✓&lt;/g&gt;</g> |
| <g>&gt;✓&lt;/g&gt;</g> | <g>&gt;✓&lt;/g&gt;</g> | <g>&gt;✓&lt;/g&gt;</g> | <g>&gt;✓&lt;/g&gt;</g> | <g>&gt;✓&lt;/g&gt;</g> | n/a                    |
| n/a                    |                        |                        |                        |                        |                        |
| Subtraction            |                        | `sub`                  |                        | `-`                    | <g>&gt;✓&lt;/g&gt;</g> |
| <g>&gt;✓&lt;/g&gt;</g> | <g>&gt;✓&lt;/g&gt;</g> | <g>&gt;✓&lt;/g&gt;</g> | <g>&gt;✓&lt;/g&gt;</g> | <g>&gt;✓&lt;/g&gt;</g> | n/a                    |
| n/a                    |                        |                        |                        |                        |                        |
| Multiplication         |                        | `mul`                  |                        | `*`                    | <g>&gt;✓&lt;/g&gt;</g> |
| <g>&gt;✓&lt;/g&gt;</g> | <g>&gt;✓&lt;/g&gt;</g> | <g>&gt;✓&lt;/g&gt;</g> | <g>&gt;✓&lt;/g&gt;</g> | <g>&gt;✓&lt;/g&gt;</g> | n/a                    |

[illegible]

|          |         |          |  |          |           |          |     |          |
|----------|---------|----------|--|----------|-----------|----------|-----|----------|
| <g>✓</g> |         | <g>✓</g> |  | <g>✓</g> |           | <g>✓</g> |     | n/a      |
|          | n/a     |          |  |          |           |          |     |          |
|          | Max     |          |  |          | `max`     |          | n/a | <g>✓</g> |
| <g>✓</g> |         | <g>✓</g> |  | <g>✓</g> |           | <g>✓</g> |     | n/a      |
|          | n/a     |          |  |          |           |          |     |          |
|          | Not     |          |  |          | `not`     |          | n/a | <g>✓</g> |
| <g>✓</g> |         | <g>✓</g> |  | <g>✓</g> |           | <g>✓</g> |     | <g>✓</g> |
| g>       |         | n/a      |  |          |           |          |     |          |
|          | Select  |          |  |          | `select`  |          | n/a | <g>✓</g> |
| <g>✓</g> |         | <g>✓</g> |  | <g>✓</g> |           | <g>✓</g> |     | <g>✓</g> |
| g>       |         | <g>✓</g> |  |          |           |          |     |          |
|          | Decrypt |          |  |          | `decrypt` |          | n/a | <g>✓</g> |
| <g>✓</g> |         | <g>✓</g> |  | <g>✓</g> |           | <g>✓</g> |     | <g>✓</g> |
| g>       |         | <g>✓</g> |  |          |           |          |     |          |

**\*\*tip:\*\***

Division and Remainder by `0` will output with an encrypted representation of the maximal value of the uint that is used (Ex. encrypted 255 for euint8)

---

title: Inputs

sidebar\_position: 1

---

## # Receiving Encrypted Inputs

Writing confidential smart contracts, a trivial part would be to receive encrypted inputs from users:

```
```sol
function transfer(
    address to,
    InEuInt32 memory inAmount // <----- encrypted input here
) public virtual returns (euInt32 transferred) {
    euInt32 amount = FHE.asEuInt32(inAmount);
}
```
```

:::note[Important Note]

Notice in the example above the distinction between **\*\*`InEuInt32`\*\*** and **\*\*`euInt32`\*\***.

:::

## ### Input Types Conversion

The **\*\*input types\*\*** ``InEuIntxx`` (and ``InEbool``, ``InEaddress``) are special encrypted types that represent **\*\*user input\*\***. Input types contain additional information required to authenticate and validate

ciphertexts. For more on that, read on the [ZK-Verifier](../architecture/internal-utilities/verifier.md).

Before we can use an encrypted input, we need to convert it to a regular **\*\*encrypted type\*\***:

```
```sol
euint32 amount = FHE.asEuInt32(inAmount);
```
```

:::tip  
You should avoid storing encrypted input types (e.g. `InEuInt8`, etc) directly in your contract's state – it will be more expensive as these types contain more data, and it could lead to unexpected results. You should always prefer to convert it to a regular encrypted type using `FHE.asE...()`.  
:::

Now that `amount` is of type `euInt32`, we can store or manipulate it:

```
```sol
toBalance = FHE.sub(toBalance, amount);
```
```

:::tip  
Read more on the available FHE types and operations [here](../fhe-encrypted-operations.md).  
:::

### ### Full Example

```
```sol
function transfer(
    address to,
    InEuInt32 memory inAmount
) public virtual returns (euInt32 transferred) {
    euInt32 amount = FHE.asEuInt32(inAmount);

    toBalance = _balances[to];
    fromBalance = _balances[msg.sender];

    _updateBalance(to, FHE.add(toBalance, amount));
    _updateBalance(from, FHE.sub(fromBalance, amount));
}
```
```

:::note

For the example above to logically work, you will also need to manage access to the newly created ciphertexts in the `\_updateBalance()`

function. Read further on the [ACL component](acl-mechanism.md).

:::

### ### Additional Examples

#### #### Voting in a Poll

```
```solidity
    function castEncryptedVote(address poll, InEbool calldata
encryptedVote) public {
        _submitVote(poll, FHE.asEbool(encryptedVote));
    }
...`
```

Setting Encrypted User Preferences

```
```solidity
 function updateUserSetting(address user, InEuint8 calldata
encryptedSetting) public {
 _applyUserSetting(user, FHE.asEuint8(encryptedSetting));
 }
...`
```

```

sidebar_position: 4
title: Conditions
description: Understanding why if..else isn't possible with FHE and
exploring the alternatives

```

Writing code with Fully Homomorphic Encryption (FHE) introduces a fundamental shift in how we handle conditionals or branches in our code. As you already know, with FHE, we're operating on encrypted data. This means we can't use typical `if...else` branching structures, because we don't have visibility into the actual values we're comparing.

Instead, use the `select` function which works like a ternary operator (`condition ? "yes" : "no"`) but for encrypted values.

#### ## Quick Example

```
```sol
euint32 a = FHE.asEuInt32(10);
euint32 b = FHE.asEuInt32(20);
euint32 max;

// Instead of this (won't work) :
if (a.gt(b)) { // gt returns encrypted boolean (ebool), traditional
if..else will result unexpected result
```

```

    max = a;
} else {
    max = b;
}

// Do this :
ebool isHigher = a.gt(b);
max = FHE.select(isHigher, a, b);
```

```

Another reason of not using the traditional `if..else` when writing Solidity smart contracts with our FHE capabilities, is that you need to consider all possible branches of a conditional at the same execution time. It's somewhat akin to writing constant-time cryptographic code, where you want to avoid timing attacks that could leak information about secret data. (Information might be leaked by understanding if your code got inside the `if` or the `else` due to different execution times of your code branches)

To handle these conditionals, we use a concept called a "selector". A selector is a function that takes in a control and two branches, and returns the result of the branch that corresponds to the condition. A selector is like a traffic signal that decides which traffic to let through based on the color of the light (control signal).

In Fhenix, we utilize this by calling the `select` function. It's a function that takes in a condition and two inputs, and returns the input that corresponds to the state of the condition. You can think of this like a ternary boolean conditional (`condition ? "yes" : "no"`), but for encrypted data.

In the example above, `FHE.select` uses the encrypted `ebool` result of `gt`, so if `isHigher` is true (encrypted true, not a plaintext one), it returns `a`, otherwise, it returns `b`.

## ## Key Points to Remember

- All operations take place on encrypted data, so the actual values and comparison results stay concealed
- Using traditional `if..else` on encrypted data might result in **unexpected behaviour** and leak information
- The `select` function is the only way to handle conditional execution in FHE without leaking information

## ## Common Use Cases

Here are some common scenarios where you'll use `select`:

1. **Maximum/Minimum Operations**
- ```
```sol
```

```
euint32 max = FHE.select(a.gt(b), a, b);
```
```

2. ****Conditional Updates****

```
```sol  
euint32 newValue = FHE.select(shouldUpdate, newValue, currentValue);
```
```

3. ****Threshold Checks****

```
```sol  
ebool isAboveThreshold = value.gt(threshold);
euint32 result = FHE.select(isAboveThreshold, value, threshold);
```
```

Best Practices

1. Always use `select` instead of trying to implement branching logic.
2. Keep your conditional logic simple and linear
3. Remember that all operations must be performed on encrypted data
4. Consider the performance implications of complex conditional chains

sidebar_position: 1

FHERC Standards Overview

FHERC (Fully Homomorphic Encryption Request for Comments) standards define the specifications and implementations for confidential tokens and smart contracts using Fully Homomorphic Encryption (FHE) on the blockchain. These standards ensure interoperability, security, and consistency across different implementations of confidential assets.

What are FHERC Standards?

FHERC standards are similar to ERC standards in the Ethereum ecosystem but specifically designed for confidential operations. They define:

- How confidential tokens should behave
- Interface requirements for FHE-enabled smart contracts
- Methods for handling encrypted operations
- Standards for maintaining privacy while ensuring functionality

Key Features

- ****Privacy by Default****: All token operations are encrypted by default
- ****Homomorphic Operations****: Support for mathematical operations on encrypted data
- ****Standardized Interfaces****: Consistent methods across all implementations

- ****Backward Compatibility****: Integration with existing blockchain infrastructure
- ****Verifiable Privacy****: Cryptographic guarantees for confidential operations

title: FHERC20 Token Standard

sidebar_position: 2

FHERC20 Contract Documentation

Overview

The FHERC20 contract is an implementation of a Fully Homomorphic Encryption (FHE) enabled ERC20 token. It provides confidentiality for token balances through encrypted operations while maintaining compatibility with existing ERC20 infrastructure.

Core Concepts

Encrypted Balances

The FHERC20 contract stores balances as encrypted values (`euint128`) using the Fhenix CoFHE coprocessor. These encrypted balances preserve user privacy while allowing for secure operations. The actual token balance of an address is kept confidential and can only be accessed under specific conditions:

1. By the owner of the balance
2. Through sealed decryption (using a public/private key pair)
3. As part of FHE operations where the value remains encrypted throughout

Indicated Balances

To maintain compatibility with existing ERC20 infrastructure (wallets, block explorers), FHERC20 implements a concept called "indicated balances". These are public values that:

- Range from 0.0000 to 0.9999 (represented internally as 0-9999)
- Start at 0 for non-interacted accounts
- Initialize at 0.5001 upon first interaction
- Increment/decrement by 0.0001 with each transaction
- Provide visual feedback of balance changes without revealing true amounts

The indicated balance system allows users to see activity in their wallets without exposing the encrypted values.

EIP712 Permissions vs Allowances

FHERC20 removes the traditional ERC20 allowance system and instead uses EIP712 signatures for permissioned transfers:

- Each `encTransferFrom` operation requires a valid EIP712 signature
- Permissions are single-use and do not grant ongoing access to funds
- This prevents leakage of balance information while maintaining secure delegation

Contract Structure

State Variables

```
```solidity
mapping(address account => uint16) internal _indicatedBalances;
mapping(address account => euint128) private _encBalances;
```

```
uint16 internal _indicatedTotalSupply;
euint128 private _encTotalSupply;
```

```
string private _name;
string private _symbol;
uint8 private _decimals;
uint256 private _indicatorTick;
```
```

- `\_indicatedBalances`: Stores the non-encrypted indicator value for each account
- `\_encBalances`: Stores the encrypted balances for each account
- `\_indicatedTotalSupply`: The non-encrypted indicator of total supply
- `\_encTotalSupply`: The encrypted total token supply
- `\_name`, `\_symbol`, `\_decimals`: Standard ERC20 token metadata
- `\_indicatorTick`: Value used to calculate the indicator step (typically $10^{(\text{decimals}-4)}$)

EIP712 Constants

```
```solidity
bytes32 private constant PERMIT_TYPEHASH =
 keccak256(
 "Permit(address owner,address spender,uint256
value_hash,uint256 nonce,uint256 deadline)"
);
```
```

Constructor

```
```solidity
constructor(
 string memory name_,
```

```

 string memory symbol_,
 uint8 decimals_
) EIP712(name_, "1")
}

```

Initializes the token with a name, symbol, and decimal precision. Also initializes the EIP712 domain separator for permissioned transfers.

## ## Public Functions

### ### Metadata Functions

```

` ``solidity
function isFherc20() public view virtual returns (bool)
function name() public view virtual returns (string memory)
function symbol() public view virtual returns (string memory)
function decimals() public view virtual returns (uint8)
` ``

```

Standard token metadata functions, with the addition of `isFherc20()` that identifies this as an FHERC20 token.

### ### Balance and Supply Functions

```

` ``solidity
function totalSupply() public view virtual returns (uint256)
function encTotalSupply() public view virtual returns (euint128)
function balanceOfIsIndicator() public view virtual returns (bool)
function indicatorTick() public view returns (uint256)
function balanceOf(address account) public view virtual returns
(uint256)
function encBalanceOf(address account) public view virtual returns
(euint128)
` ``

```

Provides both encrypted and indicated balance/supply information:

- `totalSupply()`: Returns the indicated total supply (non-encrypted)
- `encTotalSupply()`: Returns the encrypted total supply
- `balanceOfIsIndicator()`: Always returns true to indicate this uses the indicator system
- `indicatorTick()`: Returns the value of one indicator increment
- `balanceOf()`: Returns the indicated balance for an account (non-encrypted)
- `encBalanceOf()`: Returns the encrypted balance for an account

### ### Transfer Functions

```

` ``solidity
function transfer(address, uint256) public pure returns (bool)

```

```

function encTransfer(address to, inEuint128 memory inValue) public
virtual returns (euint128 transferred)
function transferFrom(address, address, uint256) public pure returns
(bool)
function encTransferFrom(
 address from,
 address to,
 inEuint128 memory inValue,
 FHERC20_EIP712_Permit calldata permit
) public virtual returns (euint128 transferred)
```

```

The standard ERC20 `transfer` and `transferFrom` functions intentionally revert to prevent accidental use of FHERC20 tokens as standard ERC20s. They are replaced by:

- `encTransfer`: Transfers encrypted tokens using an encrypted input value
- `encTransferFrom`: Transfers tokens on behalf of another account, requiring an EIP712 signature

Allowance Functions

```

```solidity
function allowance(address, address) external pure returns (uint256)
function approve(address, uint256) external pure returns (bool)
```

```

Both functions revert with `FHERC20IncompatibleFunction()` since the standard allowance system is replaced with EIP712 permits.

EIP712 and Utility Functions

```

```solidity
function nonces(address owner) public view returns (uint256)
function DOMAIN_SEPARATOR() external view virtual returns (bytes32)
function resetIndicatedBalance() external
```

```

- `nonces`: Tracks the EIP712 nonce for each address (for replay protection)
- `DOMAIN\_SEPARATOR`: Returns the EIP712 domain separator
- `resetIndicatedBalance`: Allows a user to reset their own indicated balance to zero

Internal Functions

Core Balance Management

```

```solidity

```

```

function _transfer(address from, address to, uint128 value) internal
returns (uint128 transferred)
function _update(address from, address to, uint128 value) internal
virtual returns (uint128 transferred)
function _mint(address account, uint128 value) internal returns
(uint128 transferred)
function _burn(address account, uint128 value) internal returns
(uint128 transferred)
```

```

These functions manage token movements:

- `\_transfer`: Internal implementation of transfer logic
- `\_update`: Core function that handles all balance updates (transfers, mints, burns)
- `\_mint`: Creates new tokens and assigns them to an account
- `\_burn`: Destroys tokens from an account

Indicator Management

```

```solidity
function _incrementIndicator(uint16 current) internal pure returns
(uint16)
function _decrementIndicator(uint16 value) internal pure returns
(uint16)
```

```

Utility functions for managing the indicator values:

- `\_incrementIndicator`: Increases an indicator by 1 (representing +0.0001)
- `\_decrementIndicator`: Decreases an indicator by 1 (representing -0.0001)

Events

The contract inherits and emits standard ERC20 events:

- `Transfer(address indexed from, address indexed to, uint256 value)`: Emitted for all transfers with the indicator tick as value
- `EncTransfer(address indexed from, address indexed to, bytes32 evalue)`: Emitted with the encrypted transfer value

Security Considerations

1. **Access Control**: The contract implements FHE access controls to ensure only authorized parties can decrypt balances
2. **Overflow Protection**: Uses FHE operations that prevent overflow in encrypted values
3. **Encryption Leakage**: Carefully designed to prevent leaking

information about encrypted balances

4. ****Signature Validation****: Implements proper signature validation for EIP712 permits

Usage Notes

1. All standard ERC20 functions that would compromise privacy (`transfer`, `approve`, `transferFrom`, `allowance`) intentionally revert
2. Applications must use the `enc`-prefixed functions for actual token operations
3. The indicator system is a transitional mechanism until infrastructure better supports encrypted tokens
4. Users can opt out of the indicator system by calling `resetIndicatedBalance()`

Advanced Features

Encrypted Transfers

The contract implements a confidential transfer system where:

- Transfers use encrypted values (`inEuint128`)
- The actual amount transferred is only known to the parties involved
- The contract performs encrypted comparisons to ensure sufficient balances
- If a user attempts to transfer more than their balance, the transfer will succeed but with zero tokens moved

Access Control for Encrypted Values

The `\_update` function handles FHE access control:

```
```solidity
if (euint128.unwrap(_encBalances[from]) != 0) {
 FHE.allowThis(_encBalances[from]);
 FHE.allow(_encBalances[from], from);
 FHE.allow(transferred, from);
}
if (euint128.unwrap(_encBalances[to]) != 0) {
 FHE.allowThis(_encBalances[to]);
 FHE.allow(_encBalances[to], to);
 FHE.allow(transferred, to);
}

FHE.allow(transferred, msg.sender);
FHE.allowGlobal(_encTotalSupply);
```
```

This ensures:

- Users can access their own balances
- The contract can perform operations on balances
- Total supply is globally accessible

Handling Transfer Results

A critical aspect of FHERC20 is the need to properly handle the result of transfer operations. Due to privacy guarantees, the FHERC20 contract implements a unique pattern for transfers:

Understanding Zero-Replacement in Transfers

When a user attempts to transfer more tokens than they have, the FHERC20 contract does not revert the transaction. Instead, it:

1. Performs an encrypted comparison between the requested transfer amount and the user's balance
2. If there are insufficient funds, it replaces the transfer amount with zero
3. Returns the actual transferred amount (which could be either the requested amount or zero)

This behavior is implemented in the `\_update` function:

```
```solidity
if (from != address(0)) {
 transferred = FHE.select(
 value.lte(_encBalances[from]),
 value,
 FHE.asEuint128(0)
);
} else {
 transferred = value;
}
```
```

Two-Step Process for Transfer-Dependent Operations

If your contract needs to perform operations based on the actual transferred amount, you must use a two-step process:

1. ****First Transaction****: Perform the transfer and decrypt the result

```
```solidity
euint128 transferred = _burn(msg.sender, value);
FHE.decrypt(transferred);
```
```

2. ****Create a Claim****: Store the information about the pending

operation

```
```solidity
_createClaim(to, value, transferred);
```
```

3. ****Second Transaction****: Process the operation once the decryption result is available

```
```solidity
function claimDecrypted(uint256 ctHash) public {
 Claim memory claim = _handleClaim(ctHash);

 // Only now perform operations that depend on the actual amount
 _erc20.safeTransfer(claim.to, claim.decryptedAmount);
}
```
```

Example Implementation Pattern

The `ConfidentialERC20` contract demonstrates this pattern:

```
```solidity
function decrypt(address to, uint128 value) public {
 if (to == address(0)) revert InvalidRecipient();

 // Step 1: Perform the transfer and decrypt the result
 uint128 burned = _burn(msg.sender, value);
 FHE.decrypt(burned);

 // Step 2: Create a claim for later processing
 _createClaim(to, value, burned);
 emit DecryptedERC20(msg.sender, to, value);
}

function claimDecrypted(uint256 ctHash) public {
 // Step 3: Process the operation once decryption is complete
 Claim memory claim = _handleClaim(ctHash);

 // Send the ERC20 to the recipient using the actual decrypted
 amount
 _erc20.safeTransfer(claim.to, claim.decryptedAmount);
 emit ClaimedDecryptedERC20(msg.sender, claim.to,
 claim.decryptedAmount);
}
```
```

The `ConfidentialClaim` contract handles the claim management:

```
```solidity
```

```

function _handleClaim(uint256 ctHash) internal returns (Claim memory
claim) {
 claim = _claims[ctHash];

 // Verify the claim exists and hasn't been processed yet
 if (claim.to == address(0)) revert ClaimNotFound();
 if (claim.claimed) revert AlreadyClaimed();

 // Get the decrypted amount (only available after decryption
 completes)
 uint128 amount = SafeCast.toUint128(FHE.getDecryptResult(ctHash));

 // Update and finalize the claim
 claim.decryptedAmount = amount;
 claim.decrypted = true;
 claim.claimed = true;

 // ... further claim processing ...
}

```

### ### Important Considerations

1. **\*\*Asynchronous Processing\*\***: Decryption results are not available in the same transaction
2. **\*\*Gas Efficiency\*\***: The two-step process requires multiple transactions, which affects gas costs
3. **\*\*User Experience\*\***: Applications should account for the waiting period between steps
4. **\*\*Security\*\***: Claims must be properly tracked and validated to prevent unauthorized access
5. **\*\*Error Handling\*\***: Implement proper error handling for cases where the transferred amount is zero

This pattern ensures that confidential transfers can be safely integrated with other smart contract operations, preserving both privacy and transactional integrity.

```

sidebar_position: 1

```

### # Pay Attention

> Critical limitations and important considerations when working with CoFHE

### ## Common Issues

1. **`Missing revert data`**– In case of getting such error, please verify [here](component-compatibility.md), that you are using the latest

cofhe-contracts version.

#### \*Will be filled overtime, when a new issue will arise.

## Possible errors from Solidity

Error	Description
**InvalidInputsAmount**	Operation requires specific number of inputs. Occurs when an operation receives wrong number of arguments
**InvalidOperationInputs**	Operation inputs must be valid for the operation. Thrown when inputs violate operation requirements
**TooManyInputs**	Operations have maximum input limits. Error when input count exceeds operation's maximum
**InvalidBytesLength**	Byte arrays must match expected length. Occurs when byte array length doesn't match required size
**InvalidTypeOrSecurityZone**	Operations must use compatible types and security zones. Occurs when operation violates type or security zone constraints
**InvalidInputType**	Input must match expected type. Error when input type doesn't match function requirements
**InvalidInputForFunction**	Function inputs must match defined parameters. Thrown when function receives incompatible input type
**InvalidSecurityZone**	Operations must stay within defined security zones. Error when operation violates security zone constraints
**InvalidSignature**	Cryptographic signatures must be valid. Occurs with signature verification failures
**InvalidSigner**	Signer must match expected authorized address. Error when transaction signer doesn't match required address
**InvalidAddress**	Address must be valid and non-zero. Error when invalid address is provided
**OnlyOwnerAllowed**	Function restricted to contract owner. Error includes address of unauthorized caller
**OnlyAggregatorAllowed**	Function restricted to authorized aggregator. Error includes address of unauthorized caller
**AlreadyDelegated**	Delegatee contract is already delegatee for sender & delegator addresses. Error when attempting duplicate delegation
**SenderCannotBeDelegateeAddress**	Sender cannot be the delegatee address. Error when sender tries to delegate to themselves
**SenderNotAllowed**	Sender address not authorized for allow operations. Error includes address of unauthorized sender
**DirectAllowForbidden**	Direct handle allowance not permitted. Must use Task Manager. Error includes address attempting direct allow

---

sidebar\_position: 2

---

# Compatibility

> **\*\*Essential version information for all CoFHE ecosystem components\*\***

This document provides detailed compatibility information for all components in the CoFHE ecosystem. Following these version requirements is crucial for maintaining a stable and secure development environment.

## Core Components

Component	Current Version	Minimum Compatible Version	Notes
-----	-----	-----	-----
<b>**cofhe-contracts**</b>	<code>[`0.0.13`](https://github.com/FhenixProtocol/cofhe-contracts/tree/v0.0.13)</code>	<code>[abi](https://cofhe-docs.s3.us-east-1.amazonaws.com/v0.0.13/index.html)</code>	<code>[`0.0.13`](https://github.com/FhenixProtocol/cofhe-contracts/tree/v0.0.13)</code> <code>[abi](https://cofhe-docs.s3.us-east-1.amazonaws.com/v0.0.13/index.html)</code>
Core smart contracts for FHE operations			
<b>**cofhe.js**</b>	<code>[`0.0.1-alpha.14`](https://github.com/FhenixProtocol/cofhejs/releases/tag/v0.0.1-alpha.14)</code>	<code>[`0.0.1-alpha.14`](https://github.com/FhenixProtocol/cofhejs/releases/tag/v0.0.1-alpha.14)</code>	JavaScript library for client-side encryption

## Network Compatibility

Network	Compatible	API Version	Notes
-----	-----	-----	-----
<b>**Sepolia**</b>	✅	<code>`v1`</code>	Full support
<b>**Arbitrum Sepolia**</b>	✅	<code>`v1`</code>	Full support

> 🚀 **\*\*More Networks Coming Soon!\*\*** We're actively working on expanding network support to bring CoFHE's powerful privacy features to even more chains. Stay tuned for exciting updates!

---  
title: Best Practices  
sidebar\_position: 5  
---

# Best Practices

This guide outlines key best practices for developing with CoFHE, based on recommendations from our development team.

## Security Considerations

### ### Decrypt Carefully

- **\*\*Always consider information leakage\*\***: Before decrypting data, evaluate what information you're exposing, how your code branches based on decrypted values, and what an observer might learn.
- **\*\*Minimize decryption operations\*\***: Only decrypt when absolutely necessary and after all sensitive computations are complete.
- **\*\*Always Update Permissions\*\***:  
Remember to call ``FHE.allowThis()`` after modifying any encrypted value that needs to be accessed later:

```
```solidity
counter = FHE.add(counter, FHE.asEuint32(1));
FHE.allowThis(counter); // Required!
```
```

### ### Avoid Code Branching Based on Encrypted Data

- **\*\*Remember there is no secure code branching with FHE\*\***: Decrypting to make branching decisions is generally a bad practice.
- **\*\*Use constant-time algorithms\*\***: Design your code to follow the same execution path regardless of encrypted values.
- **\*\*Prefer FHE.select over conditional logic\*\***: Use built-in selection operations rather than decrypting for if/else decisions.  
Since conditional branching doesn't work with encrypted values, use ``FHE.select()`` instead:

```
```solidity
// Instead of: if (condition) { result = a; } else { result = b; }
result = FHE.select(condition, a, b);
```
```

[see more](../fhe-library/select-vs-ifelse.md)

## ## Performance Optimization

### ### Optimize Computational Efficiency

- **\*\*Minimize FHE operations\*\***: Each operation adds computational overhead.
- **\*\*Use the minimum bit-width necessary\*\***: Choose the smallest integer type that can safely represent your data (e.g., `euint32` instead of `euint64` when possible).
- **\*\*Reuse Encrypted Constants – gas saver\*\***:  
Encrypt constant values once and reuse them to save gas:

```
```solidity
// Good practice
euint32 ONE = FHE.asEuint32(1);
FHE.allowThis(ONE);

// Later in the code
counter = FHE.add(counter, ONE);
```
```

### ### Plan for Asynchronous Operations

- **\*\*Implement loading indicators in your UI\*\***: Due to CoFHE's asynchronous nature, operations may take time to complete.
- **\*\*Use progress indicators\*\***: Show spinners, progress bars, or status messages to inform users when operations are in progress.
- **\*\*Consider state management\*\***: Design your application to handle pending states gracefully.

---

sidebar\_position: 1

title: Quick Start Guide

description: Begin your journey with CoFHE to build privacy-preserving blockchain applications

---

## # CoFHE Quick Start Guide

Welcome to the CoFHE Quick Start Guide! This guide will take you through a carefully designed learning path to build privacy-preserving applications using Fully Homomorphic Encryption (FHE) on Ethereum and other EVM-compatible chains.

## ## Your Learning Path

We've designed this Quick Start section as a progressive journey from fundamentals to application:

### ### 1. Local Development Setup

First, you'll set up a local development environment to build and test your FHE applications.

**\*\*What you'll learn:\*\***

- How to set up a complete local Fhenix environment
- Running the necessary components with Docker
- Understanding the key components of the FHE infrastructure
- Configuring your development tools

→ [Set Up Local Development](/docs/devdocs/quick-start/local-development)

### ### 2. Your First FHE Contract

Next, you'll build a simple encrypted counter contract to understand FHE on the smart contract side.

**\*\*What you'll learn:\*\***

- How to define and work with encrypted data types
- Performing operations on encrypted values
- Access control and permissions

– Asynchronous decryption process

→ [\[Build Your First FHE Contract\]\(/docs/devdocs/quick-start/Your-First-FHE-Contract\)](#)

### ### 3. Getting Started with Cofhejs

Next, you'll learn how to use the JavaScript SDK to interact with FHE-enabled contracts.

**\*\*What you'll learn:\*\***

- The mental model behind encrypted data processing
- How to set up cofhejs in your project
- Core operations: encryption, permits, and unsealing
- Local development environment configuration

→ [\[Begin with Cofhejs Setup\]\(/docs/devdocs/quick-start/getting-started\)](#)

### ### 4. Best Practices

Finally, learn how to write secure and efficient FHE code.

**\*\*What you'll learn:\*\***

- Security considerations for FHE
- Performance optimization techniques

→ [\[Apply Best Practices\]\(/docs/devdocs/quick-start/best-practices\)](#)

## ## Next Steps After Quick Start

Once you've completed the Quick Start, continue your learning journey with:

- [\[Complete CoFHE.js Documentation\]\(/docs/category/cofhejs\)](#) – Deep dive into the JavaScript SDK
- [\[FHE Library Reference\]\(/docs/devdocs/fhe-library\)](#) – Learn about core FHE operations
- [\[Solidity API Reference\]\(/docs/devdocs/solidity-api/FHE.md\)](#) – Complete reference for the Solidity library
- [\[Advanced Tutorials\]\(/docs/category/tutorials\)](#) – Learn how to add FHE to existing contracts

Ready to start building the future of privacy-preserving applications? Let's begin!

---

title: Local Development Setup

sidebar\_position: 2

description: Setting up your local development environment with and without Threshold Network

---

## # Local Development Setup

### ## Overview

This guide explains how to set up a local development environment for Fhenix, consists of several key components working together to provide a complete development experience.

The full Fhenix local environment includes:

- **\*\*Preprocessor\*\***: Generates cryptographic keys and preprocessing data
- **\*\*CoFHE Nodes\*\***: Host chain and registry chain for blockchain functionality
- **\*\*FHEOS Server\*\***: Core service for FHE operations
- **\*\*Threshold Decryption Network\*\***: Distributed decryption network
- **\*\*ZK Verifier\*\***: Zero-knowledge verification service

### ## How to run

1. Clone the CoFHE repository (it contains all the needed config files and etc)

```
```bash
> git clone git@github.com:FhenixProtocol/cofhe.git
> cd cofhe
> docker compose up -d
```
```

2. Alternatively, you can use custom configuration paths:

```
```bash
> CONFIG_DIR=/path_to_local_configs_folder \
KEYS_PATH=/path_to_local_keys_folder \
docker compose up -d
```
```

### ## Docker compose steps

#### ### 1. Preprocessor

Generates keys and preprocessed data for the decryption threshold network.

- Automatically skips if keys already exist in the mounted folder
- To force new key creation, add `--force-creation` to the preprocessor command in docker-compose.yml:  
`preprocessor --runtime-config test --force-creation &&`
- Initial key generation takes approximately 2-4 minutes, and subsequent containers will wait for this step to complete

#### ### 2. CoFHE Nodes and Services

- Two blockchain nodes initialize while keys are being generated
- After successful key generation, the `fheos\_server` starts
- The Middleware container follows, deploying contracts and starting the Aggregator

### ### 3. Threshold Decryption Network

- RabbitMQ messaging service deploys first
- Once RabbitMQ is ready, Dispatcher and Coordinator services initialize
- After key generation completes, Party Member services (PMs) start and load keys from the shared volume

### ### 4. ZK Verifier

- Relies on the generated cryptographic keys
- Exposes the signer's public key
- Requires appropriate access permissions to the keys folder

## ## 📶 Networking 📶

> [!NOTE]

> The docker compose configuration uses `"network\_mode: host"` for simplicity.

> If this doesn't work for your environment, you can define a custom docker network and use the exposed ports specified in the docker-compose files.

## ## ⚙️ Configurations ⚙️

This local setup requires multiple configuration files, which are included in the CoFHE repository at:  
<https://github.com/FhenixProtocol/cofhe/tree/master/localcofhenix/configs>

Your configuration folder (`CONFIG\_DIR`) should contain:

1. `config.toml` - Used by Dispatcher and Coordinator
2. `pm\_config\_00-0x.toml` - Used by Party Members (each PM uses its own config file)
3. `zk\_verifier\_config.toml` - Used by the ZK Verifier

## ## 🔑 Keys 🔑

After the preprocessor container finishes successfully, your `KEYS\_PATH` (or `./localcofhenix/keys` if not specified) will contain the cryptographic keys required by the FHEOS server, Party Members, ZK Verifier, and other components.

## ## 🛠️ Troubleshooting 🛠️

- If services fail to start, check that the preprocessor completed successfully
- Some services may require a restart if they attempt to access keys

before generation is complete

- If you need to restart the Middleware container, be aware that contract redeployments may occur and break stuff. In this case:
  1. Stop the Middleware container
  2. Restart the blockchain nodes
  3. Restart the Middleware container
- Review the container logs for specific error messages using ``docker logs [container_name]``
- Ensure you have sufficient system resources (CPU, memory) for running the full stack

---

title: Getting Started with Cofhejs  
sidebar\_position: 4  
description: Setup instructions for cofhejs

---

```
import Tabs from "@theme/Tabs";
import TabItem from "@theme/TabItem";
```

## ## Overview

cofhejs is a TypeScript package designed to enable seamless interaction between clients and the Fhenix's co-processor (CoFHE). It is an essential component for engineers working with FHE-enabled smart contracts, as it facilitates the encryption and decryption processes required for secure data handling in decentralized applications (dApps). cofhejs ensures that data remains private throughout its journey from input to output in the blockchain ecosystem. FHE-enabled contracts require three primary modifications to the client/frontend:

- \* **Encrypting Input Data:** Before passing data to the smart contract, input must be encrypted to ensure its confidentiality. To read more about encrypted inputs, go [\[here\] \(/docs/devdocs/cofhejs/encryption-operations\)](/docs/devdocs/cofhejs/encryption-operations).
- \* **Creating Permits and Permissions:** The client must generate permits and permissions that determine who can interact with or view the data. Read more about [\[permits\] \(/docs/devdocs/cofhejs/permits-management\)](/docs/devdocs/cofhejs/permits-management).
- \* **Unsealing Output Data:** After the contract processes the data, the client must decrypt the output for it to be used or displayed. For more, refer to our page on [\[sealing and unsealing\] \(/docs/devdocs/cofhejs/sealing-unsealing\)](/docs/devdocs/cofhejs/sealing-unsealing).

cofhejs allows encryption to begin and end privately in a dApp, while FHE-enabled contracts do work on and with these encrypted values.

## ## Mental Model

To understand how **cofhejs** fits into the Fhenix framework, we will create a simple mental model to show how data moves through Fhenix-

powered dApps.

Consider a smart contract called "\*\*Counter\*\*". Each user has an individual counter, and users increment and read their own counters with complete privacy. In this example, a \*\*public key\*\* is like a lock, and a \*\*private key\*\* is the corresponding key to unlock it.

### ### Adding to the User's Counter

When users want to add a value to their counter, say "5," they first place this value inside a sort-of "box". Using cofhejs, this box is secured by locking it with Fhenix Co-Processor's \*\*public key\*\* (encryption). The locked box is then sent to the smart contract. Thanks to Fully Homomorphic Encryption (FHE), Fhenix can perform mathematical operations directly on these sealed boxes—without accessing the raw data inside. So, the user's encrypted value, "5," can be added to the user's encrypted counter while remaining private.

### ### Retrieving the User's Counter

To retrieve the counter value, the user needs to read the data inside the box without breaking the encryption. Here's the clever part: the user sends a second "lock" (their own public key) along with the request to read its data. This second lock is applied to the box (now the box has two locks). Fhenix can now remove its own lock (the Co-Processor's public key), leaving the box secured by only the user's public key. The box remains locked and the data remains private, but now only the user can open it using its private key.

## ## Installation

To get started with cofhejs, you need to install it as a dependency in your JavaScript project. You can do this using npm (Node Package Manager) or Yarn. Open your terminal and navigate to your project's directory, then run the following:

```
<Tabs>
 <TabItem value="yarn" label="yarn">
    ``` yarn add cofhejs ```
  </TabItem>
  <TabItem value="npm" label="npm">
    ``` npm install cofhejs ```
 </TabItem>
 <TabItem value="pnpm" label="pnpm">
    ``` pnpm add cofhejs ```
  </TabItem>
</Tabs>
```

Setup

To use **cofhejs** for interacting with FHE-enabled smart contracts, the `cofhejs` must be initialized. This client handles key operations such as encrypting input data, creating permits, and decrypting output data from the blockchain.

First, the client must be initialized. Below is an example setup:

```
<Tabs>
  <TabItem value="node" label="Node.js">
    ```javascript
 const { cofhejs } = require("cofhejs/node");
 const { ethers } = require("ethers");

 // initialize your web3 provider
 const provider = new ethers.JsonRpcProvider("http://
127.0.0.1:42069");
 const wallet = new ethers.Wallet(PRIVATE_KEY, provider);

 // initialize cofhejs Client with ethers (it also supports viem)
 await cofhejs.initializeWithEthers({
 ethersProvider: provider,
 ethersSigner: wallet,
 environment: "LOCAL",
 });
    ```
  </TabItem>
  <TabItem value="web" label="Browser">
    ```javascript
 const { cofhejs } = require("cofhejs/web");
 const { ethers } = require("ethers");

 // initialize your web3 provider
 const provider = new ethers.BrowserProvider(window.ethereum);
 const signer = (await provider.getSigner()) as
ethers.JsonRpcSigner;

 // initialize cofhejs Client with ethers (it also supports viem)
 await cofhejs.initializeWithEthers({
 ethersProvider: provider,
 ethersSigner: wallet,
 environment: "LOCAL",
 });
    ```
  </TabItem>
</Tabs>
```

Encrypting Input Data

This step secures the data before sending it to the smart contract. Remember--all data sent to a smart contract on a blockchain is inherently public, which means that anyone can see it. However, Fhenix operates differently. To maintain user confidentiality and protect sensitive input data, Fhenix utilizes **cofhejs** to provide built-in encryption methods that must be applied before sending any data to an FHE-enabled contract (Learn more [\[here\] \(/docs/devdocs/cofhejs/encryption-operations\)](/docs/devdocs/cofhejs/encryption-operations)).

In the following example, we will encrypt a multiple values and pass them to a smart contract. The function `logState` is a callback function that will be called with the current state of the encryption process.

```
```javascript
const logState = (state: EncryptStep) => {
 console.log(`Log Encrypt State :: ${state}`);
};

// This will encrypt only the encrypted values (total 4 in this case)
const encryptedValues = await cofhejs.encrypt(logState, [
 { a: Encryptable.bool(false), b: Encryptable.uint64(10n), c:
"hello" },
 ["hello", 20n, Encryptable.address(contractAddress)],
 Encryptable.uint8("10"),
] as const);

const contract = new ethers.Contract(CONTRACT_ADDRESS, CONTRACT_ABI,
wallet);
// Use the encrypted value of 10n
const tx = await contract.add(encryptedValues.data[1]);

...
```
```

By encrypting user data before sending it to a contract, Fhenix ensures that data remains private throughout its lifecycle in the blockchain environment.

Creating Permits

After encryption, values can be passed into FHE-enabled smart contracts, and the contract can operate on this data securely, within its own logic. However, to ensure that only the respective user can view the processed (encrypted) data, **permissions** and **sealing** mechanisms are used. These ensure that data remains private and viewable exclusively by the user who owns it. Learn more at [\[Permits](#)

Management](/docs/devdocs/cofhejs/permits-management) and [Sealing and Unsealing](/docs/devdocs/cofhejs/sealing-unsealing).

Permissions serve two main purposes:

- **Verify User Identity**: They ensure that the data access request comes from the correct user by verifying that the message is signed with the user's private key.
- **Sealing User Data**: They provide a **public key** to "seal" the encrypted data, meaning it is encrypted in such a way that only the user holding the corresponding **private key** (stored securely on the user's client) can decrypt it later.

Note: Fhenix uses **EIP712**, a widely used Ethereum standard for signing structured data. This means: first, a user must sign a permit in their wallet to authenticate themselves and authorize the creation of the permit; second, permits are stored locally in local storage and can be reused for future interactions with the same contract. Currently, each contract that the user interacts with requires its own unique permit (subject to change).

Here's the code for this process:

```
```javascript
const permit = await cofhejs.createPermit({
 type: "self",
 issuer: wallet.address,
});
```
```

Unsealing Data

After encryption, the data can be securely processed by the contract and sealed with the user's **public key** (from their permit), and it is returned to the user when the user requests it. To access and interpret this data, the user must **unseal** it using their private key, which is securely stored on their device. The unsealing process is essential to ensure that only the intended user can view the final result.

When the contract returns the encrypted data to the user, it remains sealed. This means the data is still encrypted with the user's **public key** and cannot be read until the corresponding **private key** is used to unlock it. **cofhejs** provides a simple method to handle this.

Here's example code to show how the unsealing process works:

```
```javascript
const permit = await cofhejs.getPermit({
 type: "self",
```

```

 issuer: wallet.address,
 });

const result = await contract.getSomeEncryptedValue();
const unsealed = await cofhejs.unseal(result, FheTypes.Uint32,
permit.data.issuer, permit.data.getHash());
```

```

End-to-End Example

This example demonstrates a full interaction between a dApp and an FHE-enabled smart contract using the `cofhejs`. It walks through how to set up the client, encrypt data, send it to the contract, create a permit for accessing sealed data, and finally unseal the returned data for the user.

```

<Tabs>
  <TabItem value="node" label="Node.js">
    ```javascript

 const { cofhejs, FheTypes } = require("cofhejs/node");
 const { ethers } = require("ethers");

 // initialize your web3 provider
 const provider = new ethers.JsonRpcProvider("http://
127.0.0.1:42069");
 const wallet = new ethers.Wallet(PRIVATE_KEY, provider);

 // initialize cofhejs Client with ethers (see cofhejs docs for
viem)
 await cofhejs.initializeWithEthers({
 ethersProvider: provider,
 ethersSigner: wallet,
 environment: "LOCAL",
 });

 const contract = new ethers.Contract(CONTRACT_ADDRESS,
CONTRACT_ABI, wallet);

 const logState = (state) => {
 console.log(`Log Encrypt State :: ${state}`);
 };

 const readCounterDecryptedValue = async () => {
 try {
 const result = await contract.get_counter_value();
 console.log("readCounterDecryptedValue result:", result);
 } catch (error) {
 console.error("Error reading from contract:", error);
 }
 };
 </TabItem>
</Tabs>

```

```

 }
}

const readCounterEncryptedValue = async () => {
 const result = await contract.get_encrypted_counter_value();
 console.log("Result:", result);

 // Let's create a permit to unseal the encrypted value
 const permit = await cofhejs.createPermit({
 type: "self",
 issuer: wallet.address,
 });

 // When creating a permit cofhejs will use it automatically,
 but you can pass it manually as well
 const unsealed = await cofhejs.unseal(result, FheTypes.Uint64,
 permit.data.issuer, permit.data.getHash());
 console.log(unsealed);
}

const incrementCounter = async () => {
 const tx = await contract.increment_counter();
 console.log("incrementCounter tx hash:", tx.hash);
 await tx.wait();
}

const resetCounter = async (value) => {
 const tx = await contract.reset_counter(value);
 console.log("resetCounter tx hash:", tx.hash);
 await tx.wait();
}

const decryptCounter = async () => {
 const tx = await contract.decrypt_counter();
 console.log("decryptCounter tx hash:", tx.hash);
 await tx.wait();
}

// Value not ready (when running this script for the first time)
await readCounterDecryptedValue();

await incrementCounter();

// Return the value 1 (after unsealing)
await readCounterEncryptedValue();

await incrementCounter();

// Sending transaction to decrypt the counter
await decryptCounter();

```

```

 // Result should be 2
 await readCounterDecryptedValue();

 const encryptedValues = await cofhejs.encrypt(logState,
[Encryptable.uint64(10n)]);
 await resetCounter(encryptedValues.data[0]);

 // Result should be 10
 await readCounterEncryptedValue();

 ...
 </TabItem>
</Tabs>

```

## ## Troubleshoot

TBD

---

title: FHE.Sol  
 sidebar\_position: 1  
 ---

## # FHE Library Documentation

### ## Encrypted Data Types

The library supports the following encrypted data types:

| Type                    | Description                        |
|-------------------------|------------------------------------|
| <code>`ebool`</code>    | Encrypted boolean value            |
| <code>`euint8`</code>   | Encrypted 8-bit unsigned integer   |
| <code>`euint16`</code>  | Encrypted 16-bit unsigned integer  |
| <code>`euint32`</code>  | Encrypted 32-bit unsigned integer  |
| <code>`euint64`</code>  | Encrypted 64-bit unsigned integer  |
| <code>`euint128`</code> | Encrypted 128-bit unsigned integer |
| <code>`euint256`</code> | Encrypted 256-bit unsigned integer |
| <code>`eaddress`</code> | Encrypted Ethereum address         |

### ## Core Functions

#### ### Type Conversion

#### asEbool (from uint/bool)  
 ```solidity  
 asEbool(bool value) → ebool

```
asEbool(bool value, int32 securityZone) → ebool
```
```

Converts a plaintext boolean value to an encrypted boolean.

```
asEuint8 (from uint)
```solidity
asEuint8(uint256 value) → euint8
asEuint8(uint256 value, int32 securityZone) → euint8
```
```

Converts a plaintext value to an encrypted 8-bit unsigned integer.

```
asEuint16 (from uint)
```solidity
asEuint16(uint256 value) → euint16
asEuint16(uint256 value, int32 securityZone) → euint16
```
```

Converts a plaintext value to an encrypted 16-bit unsigned integer.

```
asEuint32 (from uint)
```solidity
asEuint32(uint256 value) → euint32
asEuint32(uint256 value, int32 securityZone) → euint32
```
```

Converts a plaintext value to an encrypted 32-bit unsigned integer.

```
asEuint64 (from uint)
```solidity
asEuint64(uint256 value) → euint64
asEuint64(uint256 value, int32 securityZone) → euint64
```
```

Converts a plaintext value to an encrypted 64-bit unsigned integer.

```
asEuint128 (from uint)
```solidity
asEuint128(uint256 value) → euint128
asEuint128(uint256 value, int32 securityZone) → euint128
```
```

Converts a plaintext value to an encrypted 128-bit unsigned integer.

```
asEuint256 (from uint)
```solidity
asEuint256(uint256 value) → euint256
asEuint256(uint256 value, int32 securityZone) → euint256
```
```

Converts a plaintext value to an encrypted 256-bit unsigned integer.

```
asEaddress (from address)
```solidity
asEaddress(address value) → eaddress
asEaddress(address value, int32 securityZone) → eaddress
```

```

Converts a plaintext address value to an encrypted address.

```
asEbool (from InEbool)
```solidity
asEbool(InEbool memory value) → ebool
```
```

Converts an encrypted input structure to an encrypted boolean.

```
asEuint8 (from InEuint8)
```solidity
asEuint8(InEuint8 memory value) → euint8
```
```

Converts an encrypted input structure to an encrypted 8-bit unsigned integer.

```
asEuint16 (from InEuint16)
```solidity
asEuint16(InEuint16 memory value) → euint16
```
```

Converts an encrypted input structure to an encrypted 16-bit unsigned integer.

```
asEuint32 (from InEuint32)
```solidity
asEuint32(InEuint32 memory value) → euint32
```
```

Converts an encrypted input structure to an encrypted 32-bit unsigned integer.

```
asEuint64 (from InEuint64)
```solidity
asEuint64(InEuint64 memory value) → euint64
```
```

Converts an encrypted input structure to an encrypted 64-bit unsigned integer.

```
asEuint128 (from InEuint128)
```solidity
asEuint128(InEuint128 memory value) → euint128
```
```

Converts an encrypted input structure to an encrypted 128-bit unsigned integer.

```
asEuint256 (from InEuint256)
```solidity
asEuint256(InEuint256 memory value) → euint256
```
```

Converts an encrypted input structure to an encrypted 256-bit unsigned integer.

```
asEaddress (from InEaddress)
```

```
```solidity
```

```
asEaddress(InEaddress memory value) → eaddress
```

```
```
```

Converts an encrypted input structure to an encrypted address.

### ### Type Conversion Between Encrypted Types

```
asEbool (from euint)
```

```
```solidity
```

```
asEbool(euint8 value) → ebool
```

```
asEbool(euint16 value) → ebool
```

```
asEbool(euint32 value) → ebool
```

```
asEbool(euint64 value) → ebool
```

```
asEbool(euint128 value) → ebool
```

```
asEbool(euint256 value) → ebool
```

```
asEbool(eaddress value) → ebool
```

```
```
```

Converts an encrypted integer to an encrypted boolean.

```
asEuInt8 (from other encrypted types)
```

```
```solidity
```

```
asEuInt8(ebool value) → euint8
```

```
asEuInt8(euint16 value) → euint8
```

```
asEuInt8(euint32 value) → euint8
```

```
asEuInt8(euint64 value) → euint8
```

```
asEuInt8(euint128 value) → euint8
```

```
asEuInt8(euint256 value) → euint8
```

```
asEuInt8(eaddress value) → euint8
```

```
```
```

Converts various encrypted types to an encrypted 8-bit unsigned integer.

```
asEuInt16 (from other encrypted types)
```

```
```solidity
```

```
asEuInt16(ebool value) → euint16
```

```
asEuInt16(euint8 value) → euint16
```

```
asEuInt16(euint32 value) → euint16
```

```
asEuInt16(euint64 value) → euint16
```

```
asEuInt16(euint128 value) → euint16
```

```
asEuInt16(euint256 value) → euint16
```

```
asEuInt16(eaddress value) → euint16
```

```
```
```

Converts various encrypted types to an encrypted 16-bit unsigned integer.

```
asEuInt32 (from other encrypted types)
```

```
```solidity
```

```
asEuInt32(ebool value) → euint32
```

```
asEuint32(euint8 value) → euint32
asEuint32(euint16 value) → euint32
asEuint32(euint64 value) → euint32
asEuint32(euint128 value) → euint32
asEuint32(euint256 value) → euint32
asEuint32(eaddress value) → euint32
```
```

Converts various encrypted types to an encrypted 32-bit unsigned integer.

```
asEuint64 (from other encrypted types)
```

```
```solidity
asEuint64(ebool value) → euint64
asEuint64(euint8 value) → euint64
asEuint64(euint16 value) → euint64
asEuint64(euint32 value) → euint64
asEuint64(euint128 value) → euint64
asEuint64(euint256 value) → euint64
asEuint64(eaddress value) → euint64
```
```

Converts various encrypted types to an encrypted 64-bit unsigned integer.

```
asEuint128 (from other encrypted types)
```

```
```solidity
asEuint128(ebool value) → euint128
asEuint128(euint8 value) → euint128
asEuint128(euint16 value) → euint128
asEuint128(euint32 value) → euint128
asEuint128(euint64 value) → euint128
asEuint128(euint256 value) → euint128
asEuint128(eaddress value) → euint128
```
```

Converts various encrypted types to an encrypted 128-bit unsigned integer.

```
asEuint256 (from other encrypted types)
```

```
```solidity
asEuint256(ebool value) → euint256
asEuint256(euint8 value) → euint256
asEuint256(euint16 value) → euint256
asEuint256(euint32 value) → euint256
asEuint256(euint64 value) → euint256
asEuint256(euint128 value) → euint256
asEuint256(eaddress value) → euint256
```
```

Converts various encrypted types to an encrypted 256-bit unsigned integer.

```
asEaddress (from euint256)
```

```
```solidity
asEAddress(euint256 value) → eaddress
```
```

Converts an encrypted 256-bit unsigned integer to an encrypted address.

### ### Arithmetic Operations

```
add (euint8)
```solidity
add(euint8 lhs, euint8 rhs) → euint8
```
```

Performs addition of two encrypted 8-bit unsigned integers.

```
add (euint16)
```solidity
add(euint16 lhs, euint16 rhs) → euint16
```
```

Performs addition of two encrypted 16-bit unsigned integers.

```
add (euint32)
```solidity
add(euint32 lhs, euint32 rhs) → euint32
```
```

Performs addition of two encrypted 32-bit unsigned integers.

```
add (euint64)
```solidity
add(euint64 lhs, euint64 rhs) → euint64
```
```

Performs addition of two encrypted 64-bit unsigned integers.

```
add (euint128)
```solidity
add(euint128 lhs, euint128 rhs) → euint128
```
```

Performs addition of two encrypted 128-bit unsigned integers.

```
add (euint256)
```solidity
add(euint256 lhs, euint256 rhs) → euint256
```
```

Performs addition of two encrypted 256-bit unsigned integers.

```
sub (euint8)
```solidity
sub(euint8 lhs, euint8 rhs) → euint8
```
```

Performs subtraction of two encrypted 8-bit unsigned integers.

```
sub (euint16)
```solidity
sub(euint16 lhs, euint16 rhs) → euint16
```
```

Performs subtraction of two encrypted 16-bit unsigned integers.

```
sub (euint32)
```solidity
sub(euint32 lhs, euint32 rhs) → euint32
```
```

Performs subtraction of two encrypted 32-bit unsigned integers.

```
sub (euint64)
```solidity
sub(euint64 lhs, euint64 rhs) → euint64
```
```

Performs subtraction of two encrypted 64-bit unsigned integers.

```
sub (euint128)
```solidity
sub(euint128 lhs, euint128 rhs) → euint128
```
```

Performs subtraction of two encrypted 128-bit unsigned integers.

```
sub (euint256)
```solidity
sub(euint256 lhs, euint256 rhs) → euint256
```
```

Performs subtraction of two encrypted 256-bit unsigned integers.

```
mul (euint8)
```solidity
mul(euint8 lhs, euint8 rhs) → euint8
```
```

Performs multiplication of two encrypted 8-bit unsigned integers.

```
mul (euint16)
```solidity
mul(euint16 lhs, euint16 rhs) → euint16
```
```

Performs multiplication of two encrypted 16-bit unsigned integers.

```
mul (euint32)
```solidity
mul(euint32 lhs, euint32 rhs) → euint32
```
```

Performs multiplication of two encrypted 32-bit unsigned integers.

```
mul (euint64)
```solidity
```

```
mul(euint64 lhs, euint64 rhs) → euint64
```
```

Performs multiplication of two encrypted 64-bit unsigned integers.

```
mul (euint128)
```solidity
mul(euint128 lhs, euint128 rhs) → euint128
```
```

Performs multiplication of two encrypted 128-bit unsigned integers.

```
mul (euint256)
```solidity
mul(euint256 lhs, euint256 rhs) → euint256
```
```

Performs multiplication of two encrypted 256-bit unsigned integers.

```
div (euint8)
```solidity
div(euint8 lhs, euint8 rhs) → euint8
```
```

Performs division of two encrypted 8-bit unsigned integers.

```
div (euint16)
```solidity
div(euint16 lhs, euint16 rhs) → euint16
```
```

Performs division of two encrypted 16-bit unsigned integers.

```
div (euint32)
```solidity
div(euint32 lhs, euint32 rhs) → euint32
```
```

Performs division of two encrypted 32-bit unsigned integers.

```
div (euint64)
```solidity
div(euint64 lhs, euint64 rhs) → euint64
```
```

Performs division of two encrypted 64-bit unsigned integers.

```
div (euint128)
```solidity
div(euint128 lhs, euint128 rhs) → euint128
```
```

Performs division of two encrypted 128-bit unsigned integers.

```
div (euint256)
```solidity
div(euint256 lhs, euint256 rhs) → euint256
```
```

Performs division of two encrypted 256-bit unsigned integers.

```
rem (euint8)
```solidity
rem(euint8 lhs, euint8 rhs) → euint8
```
```

Calculates the remainder when dividing two encrypted 8-bit unsigned integers.

```
rem (euint16)
```solidity
rem(euint16 lhs, euint16 rhs) → euint16
```
```

Calculates the remainder when dividing two encrypted 16-bit unsigned integers.

```
rem (euint32)
```solidity
rem(euint32 lhs, euint32 rhs) → euint32
```
```

Calculates the remainder when dividing two encrypted 32-bit unsigned integers.

```
rem (euint64)
```solidity
rem(euint64 lhs, euint64 rhs) → euint64
```
```

Calculates the remainder when dividing two encrypted 64-bit unsigned integers.

```
rem (euint128)
```solidity
rem(euint128 lhs, euint128 rhs) → euint128
```
```

Calculates the remainder when dividing two encrypted 128-bit unsigned integers.

```
rem (euint256)
```solidity
rem(euint256 lhs, euint256 rhs) → euint256
```
```

Calculates the remainder when dividing two encrypted 256-bit unsigned integers.

```
square (euint8)
```solidity
square(euint8 value) → euint8
```
```

Calculates the square of an encrypted 8-bit unsigned integer.

```
square (euint16)
```solidity
square(euint16 value) → euint16
```
```

Calculates the square of an encrypted 16-bit unsigned integer.

```
square (euint32)
```solidity
square(euint32 value) → euint32
```
```

Calculates the square of an encrypted 32-bit unsigned integer.

```
square (euint64)
```solidity
square(euint64 value) → euint64
```
```

Calculates the square of an encrypted 64-bit unsigned integer.

```
square (euint128)
```solidity
square(euint128 value) → euint128
```
```

Calculates the square of an encrypted 128-bit unsigned integer.

```
square (euint256)
```solidity
square(euint256 value) → euint256
```
```

Calculates the square of an encrypted 256-bit unsigned integer.

### ### Bitwise Operations

```
and (ebool)
```solidity
and(ebool lhs, ebool rhs) → ebool
```
```

Performs a bitwise AND operation on two encrypted boolean values.

```
and (euint8)
```solidity
and(euint8 lhs, euint8 rhs) → euint8
```
```

Performs a bitwise AND operation on two encrypted 8-bit unsigned integers.

```
and (euint16)
```solidity
and(euint16 lhs, euint16 rhs) → euint16
```
```

Performs a bitwise AND operation on two encrypted 16-bit unsigned

integers.

```
and (euint32)
```solidity
and(euint32 lhs, euint32 rhs) → euint32
```
```

Performs a bitwise AND operation on two encrypted 32-bit unsigned integers.

```
and (euint64)
```solidity
and(euint64 lhs, euint64 rhs) → euint64
```
```

Performs a bitwise AND operation on two encrypted 64-bit unsigned integers.

```
and (euint128)
```solidity
and(euint128 lhs, euint128 rhs) → euint128
```
```

Performs a bitwise AND operation on two encrypted 128-bit unsigned integers.

```
and (euint256)
```solidity
and(euint256 lhs, euint256 rhs) → euint256
```
```

Performs a bitwise AND operation on two encrypted 256-bit unsigned integers.

```
or (ebool)
```solidity
or(ebool lhs, ebool rhs) → ebool
```
```

Performs a bitwise OR operation on two encrypted boolean values.

```
or (euint8)
```solidity
or(euint8 lhs, euint8 rhs) → euint8
```
```

Performs a bitwise OR operation on two encrypted 8-bit unsigned integers.

```
or (euint16)
```solidity
or(euint16 lhs, euint16 rhs) → euint16
```
```

Performs a bitwise OR operation on two encrypted 16-bit unsigned integers.

```
or (euint32)
```solidity
or(euint32 lhs, euint32 rhs) → euint32
```
```

Performs a bitwise OR operation on two encrypted 32-bit unsigned integers.

```
or (euint64)
```solidity
or(euint64 lhs, euint64 rhs) → euint64
```
```

Performs a bitwise OR operation on two encrypted 64-bit unsigned integers.

```
or (euint128)
```solidity
or(euint128 lhs, euint128 rhs) → euint128
```
```

Performs a bitwise OR operation on two encrypted 128-bit unsigned integers.

```
or (euint256)
```solidity
or(euint256 lhs, euint256 rhs) → euint256
```
```

Performs a bitwise OR operation on two encrypted 256-bit unsigned integers.

```
xor (ebool)
```solidity
xor(ebool lhs, ebool rhs) → ebool
```
```

Performs a bitwise XOR operation on two encrypted boolean values.

```
xor (euint8)
```solidity
xor(euint8 lhs, euint8 rhs) → euint8
```
```

Performs a bitwise XOR operation on two encrypted 8-bit unsigned integers.

```
xor (euint16)
```solidity
xor(euint16 lhs, euint16 rhs) → euint16
```
```

Performs a bitwise XOR operation on two encrypted 16-bit unsigned integers.

```
xor (euint32)
```solidity
```

```
xor(euint32 lhs, euint32 rhs) → euint32
```
```

Performs a bitwise XOR operation on two encrypted 32-bit unsigned integers.

```
xor (euint64)
```solidity
xor(euint64 lhs, euint64 rhs) → euint64
```
```

Performs a bitwise XOR operation on two encrypted 64-bit unsigned integers.

```
xor (euint128)
```solidity
xor(euint128 lhs, euint128 rhs) → euint128
```
```

Performs a bitwise XOR operation on two encrypted 128-bit unsigned integers.

```
xor (euint256)
```solidity
xor(euint256 lhs, euint256 rhs) → euint256
```
```

Performs a bitwise XOR operation on two encrypted 256-bit unsigned integers.

```
not (ebool)
```solidity
not(ebool value) → ebool
```
```

Performs a bitwise NOT operation on an encrypted boolean value.

```
not (euint8)
```solidity
not(euint8 value) → euint8
```
```

Performs a bitwise NOT operation on an encrypted 8-bit unsigned integer.

```
not (euint16)
```solidity
not(euint16 value) → euint16
```
```

Performs a bitwise NOT operation on an encrypted 16-bit unsigned integer.

```
not (euint32)
```solidity
not(euint32 value) → euint32
```
```

Performs a bitwise NOT operation on an encrypted 32-bit unsigned integer.

```
not (euint64)
```solidity
not(euint64 value) → euint64
```
```

Performs a bitwise NOT operation on an encrypted 64-bit unsigned integer.

```
not (euint128)
```solidity
not(euint128 value) → euint128
```
```

Performs a bitwise NOT operation on an encrypted 128-bit unsigned integer.

```
not (euint256)
```solidity
not(euint256 value) → euint256
```
```

Performs a bitwise NOT operation on an encrypted 256-bit unsigned integer.

```
shl (euint8)
```solidity
shl(euint8 lhs, euint8 rhs) → euint8
```
```

Performs a left shift operation on an encrypted 8-bit unsigned integer.

```
shl (euint16)
```solidity
shl(euint16 lhs, euint16 rhs) → euint16
```
```

Performs a left shift operation on an encrypted 16-bit unsigned integer.

```
shl (euint32)
```solidity
shl(euint32 lhs, euint32 rhs) → euint32
```
```

Performs a left shift operation on an encrypted 32-bit unsigned integer.

```
shl (euint64)
```solidity
shl(euint64 lhs, euint64 rhs) → euint64
```
```

Performs a left shift operation on an encrypted 64-bit unsigned

integer.

```
shl (euint128)
```solidity
shl(euint128 lhs, euint128 rhs) → euint128
```
```

Performs a left shift operation on an encrypted 128-bit unsigned integer.

```
shl (euint256)
```solidity
shl(euint256 lhs, euint256 rhs) → euint256
```
```

Performs a left shift operation on an encrypted 256-bit unsigned integer.

```
shr (euint8)
```solidity
shr(euint8 lhs, euint8 rhs) → euint8
```
```

Performs a right shift operation on an encrypted 8-bit unsigned integer.

```
shr (euint16)
```solidity
shr(euint16 lhs, euint16 rhs) → euint16
```
```

Performs a right shift operation on an encrypted 16-bit unsigned integer.

```
shr (euint32)
```solidity
shr(euint32 lhs, euint32 rhs) → euint32
```
```

Performs a right shift operation on an encrypted 32-bit unsigned integer.

```
shr (euint64)
```solidity
shr(euint64 lhs, euint64 rhs) → euint64
```
```

Performs a right shift operation on an encrypted 64-bit unsigned integer.

```
shr (euint128)
```solidity
shr(euint128 lhs, euint128 rhs) → euint128
```
```

Performs a right shift operation on an encrypted 128-bit unsigned integer.

```
shr (euint256)
```solidity
shr(euint256 lhs, euint256 rhs) → euint256
```
```

Performs a right shift operation on an encrypted 256-bit unsigned integer.

```
rol (euint8)
```solidity
rol(euint8 lhs, euint8 rhs) → euint8
```
```

Performs a rotate left operation on an encrypted 8-bit unsigned integer.

```
rol (euint16)
```solidity
rol(euint16 lhs, euint16 rhs) → euint16
```
```

Performs a rotate left operation on an encrypted 16-bit unsigned integer.

```
rol (euint32)
```solidity
rol(euint32 lhs, euint32 rhs) → euint32
```
```

Performs a rotate left operation on an encrypted 32-bit unsigned integer.

```
rol (euint64)
```solidity
rol(euint64 lhs, euint64 rhs) → euint64
```
```

Performs a rotate left operation on an encrypted 64-bit unsigned integer.

```
rol (euint128)
```solidity
rol(euint128 lhs, euint128 rhs) → euint128
```
```

Performs a rotate left operation on an encrypted 128-bit unsigned integer.

```
rol (euint256)
```solidity
rol(euint256 lhs, euint256 rhs) → euint256
```
```

Performs a rotate left operation on an encrypted 256-bit unsigned integer.

```
ror (euint8)
```solidity
ror(euint8 lhs, euint8 rhs) → euint8
```
```

Performs a rotate right operation on an encrypted 8-bit unsigned integer.

```
ror (euint16)
```solidity
ror(euint16 lhs, euint16 rhs) → euint16
```
```

Performs a rotate right operation on an encrypted 16-bit unsigned integer.

```
ror (euint32)
```solidity
ror(euint32 lhs, euint32 rhs) → euint32
```
```

Performs a rotate right operation on an encrypted 32-bit unsigned integer.

```
ror (euint64)
```solidity
ror(euint64 lhs, euint64 rhs) → euint64
```
```

Performs a rotate right operation on an encrypted 64-bit unsigned integer.

```
ror (euint128)
```solidity
ror(euint128 lhs, euint128 rhs) → euint128
```
```

Performs a rotate right operation on an encrypted 128-bit unsigned integer.

```
ror (euint256)
```solidity
ror(euint256 lhs, euint256 rhs) → euint256
```
```

Performs a rotate right operation on an encrypted 256-bit unsigned integer.

### ### Comparison Operations

```
eq (ebool)
```solidity
eq(ebool lhs, ebool rhs) → ebool
```
```

Checks if two encrypted boolean values are equal and returns an encrypted boolean.

```
eq (euint8)
```solidity
eq(euint8 lhs, euint8 rhs) → ebool
```
```

Checks if two encrypted 8-bit unsigned integers are equal and returns an encrypted boolean.

```
eq (euint16)
```solidity
eq(euint16 lhs, euint16 rhs) → ebool
```
```

Checks if two encrypted 16-bit unsigned integers are equal and returns an encrypted boolean.

```
eq (euint32)
```solidity
eq(euint32 lhs, euint32 rhs) → ebool
```
```

Checks if two encrypted 32-bit unsigned integers are equal and returns an encrypted boolean.

```
eq (euint64)
```solidity
eq(euint64 lhs, euint64 rhs) → ebool
```
```

Checks if two encrypted 64-bit unsigned integers are equal and returns an encrypted boolean.

```
eq (euint128)
```solidity
eq(euint128 lhs, euint128 rhs) → ebool
```
```

Checks if two encrypted 128-bit unsigned integers are equal and returns an encrypted boolean.

```
eq (euint256)
```solidity
eq(euint256 lhs, euint256 rhs) → ebool
```
```

Checks if two encrypted 256-bit unsigned integers are equal and returns an encrypted boolean.

```
eq (eaddress)
```solidity
eq(eaddress lhs, eaddress rhs) → ebool
```
```

Checks if two encrypted addresses are equal and returns an encrypted boolean.

```
ne (ebool)
```solidity
ne(ebool lhs, ebool rhs) → ebool
```
```

Checks if two encrypted boolean values are not equal and returns an encrypted boolean.

```
ne (euint8)
```solidity
ne(euint8 lhs, euint8 rhs) → ebool
```
```

Checks if two encrypted 8-bit unsigned integers are not equal and returns an encrypted boolean.

```
ne (euint16)
```solidity
ne(euint16 lhs, euint16 rhs) → ebool
```
```

Checks if two encrypted 16-bit unsigned integers are not equal and returns an encrypted boolean.

```
ne (euint32)
```solidity
ne(euint32 lhs, euint32 rhs) → ebool
```
```

Checks if two encrypted 32-bit unsigned integers are not equal and returns an encrypted boolean.

```
ne (euint64)
```solidity
ne(euint64 lhs, euint64 rhs) → ebool
```
```

Checks if two encrypted 64-bit unsigned integers are not equal and returns an encrypted boolean.

```
ne (euint128)
```solidity
ne(euint128 lhs, euint128 rhs) → ebool
```
```

Checks if two encrypted 128-bit unsigned integers are not equal and returns an encrypted boolean.

```
ne (euint256)
```solidity
ne(euint256 lhs, euint256 rhs) → ebool
```
```

Checks if two encrypted 256-bit unsigned integers are not equal and returns an encrypted boolean.

```
ne (eaddress)
```

```
```solidity
ne(eaddress lhs, eaddress rhs) → ebool
```
```

Checks if two encrypted addresses are not equal and returns an encrypted boolean.

```
lt (euint8)
```solidity
lt(euint8 lhs, euint8 rhs) → ebool
```
```

Checks if the first encrypted 8-bit unsigned integer is less than the second and returns an encrypted boolean.

```
lt (euint16)
```solidity
lt(euint16 lhs, euint16 rhs) → ebool
```
```

Checks if the first encrypted 16-bit unsigned integer is less than the second and returns an encrypted boolean.

```
lt (euint32)
```solidity
lt(euint32 lhs, euint32 rhs) → ebool
```
```

Checks if the first encrypted 32-bit unsigned integer is less than the second and returns an encrypted boolean.

```
lt (euint64)
```solidity
lt(euint64 lhs, euint64 rhs) → ebool
```
```

Checks if the first encrypted 64-bit unsigned integer is less than the second and returns an encrypted boolean.

```
lt (euint128)
```solidity
lt(euint128 lhs, euint128 rhs) → ebool
```
```

Checks if the first encrypted 128-bit unsigned integer is less than the second and returns an encrypted boolean.

```
lt (euint256)
```solidity
lt(euint256 lhs, euint256 rhs) → ebool
```
```

Checks if the first encrypted 256-bit unsigned integer is less than the second and returns an encrypted boolean.

```
lte (euint8)
```solidity
```

```
lte(euint8 lhs, euint8 rhs) → ebool
```
```

Checks if the first encrypted 8-bit unsigned integer is less than or equal to the second and returns an encrypted boolean.

```
lte (euint16)
```solidity
lte(euint16 lhs, euint16 rhs) → ebool
```
```

Checks if the first encrypted 16-bit unsigned integer is less than or equal to the second and returns an encrypted boolean.

```
lte (euint32)
```solidity
lte(euint32 lhs, euint32 rhs) → ebool
```
```

Checks if the first encrypted 32-bit unsigned integer is less than or equal to the second and returns an encrypted boolean.

```
lte (euint64)
```solidity
lte(euint64 lhs, euint64 rhs) → ebool
```
```

Checks if the first encrypted 64-bit unsigned integer is less than or equal to the second and returns an encrypted boolean.

```
lte (euint128)
```solidity
lte(euint128 lhs, euint128 rhs) → ebool
```
```

Checks if the first encrypted 128-bit unsigned integer is less than or equal to the second and returns an encrypted boolean.

```
lte (euint256)
```solidity
lte(euint256 lhs, euint256 rhs) → ebool
```
```

Checks if the first encrypted 256-bit unsigned integer is less than or equal to the second and returns an encrypted boolean.

```
gt (euint8)
```solidity
gt(euint8 lhs, euint8 rhs) → ebool
```
```

Checks if the first encrypted 8-bit unsigned integer is greater than the second and returns an encrypted boolean.

```
gt (euint16)
```solidity
gt(euint16 lhs, euint16 rhs) → ebool
```

```

Checks if the first encrypted 16-bit unsigned integer is greater than the second and returns an encrypted boolean.

```
gt (euint32)
```solidity
gt(euint32 lhs, euint32 rhs) → ebool
```
```

Checks if the first encrypted 32-bit unsigned integer is greater than the second and returns an encrypted boolean.

```
gt (euint64)
```solidity
gt(euint64 lhs, euint64 rhs) → ebool
```
```

Checks if the first encrypted 64-bit unsigned integer is greater than the second and returns an encrypted boolean.

```
gt (euint128)
```solidity
gt(euint128 lhs, euint128 rhs) → ebool
```
```

Checks if the first encrypted 128-bit unsigned integer is greater than the second and returns an encrypted boolean.

```
gt (euint256)
```solidity
gt(euint256 lhs, euint256 rhs) → ebool
```
```

Checks if the first encrypted 256-bit unsigned integer is greater than the second and returns an encrypted boolean.

```
gte (euint8)
```solidity
gte(euint8 lhs, euint8 rhs) → ebool
```
```

Checks if the first encrypted 8-bit unsigned integer is greater than or equal to the second and returns an encrypted boolean.

```
gte (euint16)
```solidity
gte(euint16 lhs, euint16 rhs) → ebool
```
```

Checks if the first encrypted 16-bit unsigned integer is greater than or equal to the second and returns an encrypted boolean.

```
gte (euint32)
```solidity
gte(euint32 lhs, euint32 rhs) → ebool
```
```

Checks if the first encrypted 32-bit unsigned integer is greater than or equal to the second and returns an encrypted boolean.

```
gte (euint64)
```solidity
gte(euint64 lhs, euint64 rhs) → ebool
```
```

Checks if the first encrypted 64-bit unsigned integer is greater than or equal to the second and returns an encrypted boolean.

```
gte (euint128)
```solidity
gte(euint128 lhs, euint128 rhs) → ebool
```
```

Checks if the first encrypted 128-bit unsigned integer is greater than or equal to the second and returns an encrypted boolean.

```
gte (euint256)
```solidity
gte(euint256 lhs, euint256 rhs) → ebool
```
```

Checks if the first encrypted 256-bit unsigned integer is greater than or equal to the second and returns an encrypted boolean.

### ### Min/Max Functions

```
min (euint8)
```solidity
min(euint8 lhs, euint8 rhs) → euint8
```
```

Returns the smaller of two encrypted 8-bit unsigned integers.

```
min (euint16)
```solidity
min(euint16 lhs, euint16 rhs) → euint16
```
```

Returns the smaller of two encrypted 16-bit unsigned integers.

```
min (euint32)
```solidity
min(euint32 lhs, euint32 rhs) → euint32
```
```

Returns the smaller of two encrypted 32-bit unsigned integers.

```
min (euint64)
```solidity
min(euint64 lhs, euint64 rhs) → euint64
```
```

Returns the smaller of two encrypted 64-bit unsigned integers.

```
min (euint128)
```solidity
min(euint128 lhs, euint128 rhs) → euint128
```
```

Returns the smaller of two encrypted 128-bit unsigned integers.

```
min (euint256)
```solidity
min(euint256 lhs, euint256 rhs) → euint256
```
```

Returns the smaller of two encrypted 256-bit unsigned integers.

```
max (euint8)
```solidity
max(euint8 lhs, euint8 rhs) → euint8
```
```

Returns the larger of two encrypted 8-bit unsigned integers.

```
max (euint16)
```solidity
max(euint16 lhs, euint16 rhs) → euint16
```
```

Returns the larger of two encrypted 16-bit unsigned integers.

```
max (euint32)
```solidity
max(euint32 lhs, euint32 rhs) → euint32
```
```

Returns the larger of two encrypted 32-bit unsigned integers.

```
max (euint64)
```solidity
max(euint64 lhs, euint64 rhs) → euint64
```
```

Returns the larger of two encrypted 64-bit unsigned integers.

```
max (euint128)
```solidity
max(euint128 lhs, euint128 rhs) → euint128
```
```

Returns the larger of two encrypted 128-bit unsigned integers.

```
max (euint256)
```solidity
max(euint256 lhs, euint256 rhs) → euint256
```
```

Returns the larger of two encrypted 256-bit unsigned integers.

### Control Flow

```
select (euint8)
```

```
```solidity
```

```
select(ebool condition, euint8 ifTrue, euint8 ifFalse) → euint8
```

```
```
```

Conditionally selects between two encrypted 8-bit unsigned integers based on an encrypted boolean condition.

```
select (euint16)
```

```
```solidity
```

```
select(ebool condition, euint16 ifTrue, euint16 ifFalse) → euint16
```

```
```
```

Conditionally selects between two encrypted 16-bit unsigned integers based on an encrypted boolean condition.

```
select (euint32)
```

```
```solidity
```

```
select(ebool condition, euint32 ifTrue, euint32 ifFalse) → euint32
```

```
```
```

Conditionally selects between two encrypted 32-bit unsigned integers based on an encrypted boolean condition.

```
select (euint64)
```

```
```solidity
```

```
select(ebool condition, euint64 ifTrue, euint64 ifFalse) → euint64
```

```
```
```

Conditionally selects between two encrypted 64-bit unsigned integers based on an encrypted boolean condition.

```
select (euint128)
```

```
```solidity
```

```
select(ebool condition, euint128 ifTrue, euint128 ifFalse) → euint128
```

```
```
```

Conditionally selects between two encrypted 128-bit unsigned integers based on an encrypted boolean condition.

```
select (euint256)
```

```
```solidity
```

```
select(ebool condition, euint256 ifTrue, euint256 ifFalse) → euint256
```

```
```
```

Conditionally selects between two encrypted 256-bit unsigned integers based on an encrypted boolean condition.

```
select (ebool)
```

```
```solidity
```

```
select(ebool condition, ebool ifTrue, ebool ifFalse) → ebool
```

```
```
```

Conditionally selects between two encrypted boolean values based on an encrypted boolean condition.

```
select (eaddress)
```

```
```solidity
select(ebool condition, eaddress ifTrue, eaddress ifFalse) → eaddress
```
```

Conditionally selects between two encrypted addresses based on an encrypted boolean condition.

### ### Encryption and Decryption

```
encrypt (bool)
```solidity
encrypt(bool value) → ebool
```
```

Encrypts a plaintext boolean value.

```
encrypt (uint8)
```solidity
encrypt(uint8 value) → euint8
```
```

Encrypts a plaintext 8-bit unsigned integer.

```
encrypt (uint16)
```solidity
encrypt(uint16 value) → euint16
```
```

Encrypts a plaintext 16-bit unsigned integer.

```
encrypt (uint32)
```solidity
encrypt(uint32 value) → euint32
```
```

Encrypts a plaintext 32-bit unsigned integer.

```
encrypt (uint64)
```solidity
encrypt(uint64 value) → euint64
```
```

Encrypts a plaintext 64-bit unsigned integer.

```
encrypt (uint128)
```solidity
encrypt(uint128 value) → euint128
```
```

Encrypts a plaintext 128-bit unsigned integer.

```
encrypt (uint256)
```solidity
encrypt(uint256 value) → euint256
```
```

Encrypts a plaintext 256-bit unsigned integer.

```
encrypt (address)
```solidity
encrypt(address value) → eaddress
```
```

Encrypts a plaintext Ethereum address.

```
decrypt (ebool)
```solidity
decrypt(ebool value) → bool
```
```

Decrypts an encrypted boolean value. The caller must have permission to access the encrypted value.

```
decrypt (euint8)
```solidity
decrypt(euint8 value) → uint8
```
```

Decrypts an encrypted 8-bit unsigned integer. The caller must have permission to access the encrypted value.

```
decrypt (euint16)
```solidity
decrypt(euint16 value) → uint16
```
```

Decrypts an encrypted 16-bit unsigned integer. The caller must have permission to access the encrypted value.

```
decrypt (euint32)
```solidity
decrypt(euint32 value) → uint32
```
```

Decrypts an encrypted 32-bit unsigned integer. The caller must have permission to access the encrypted value.

```
decrypt (euint64)
```solidity
decrypt(euint64 value) → uint64
```
```

Decrypts an encrypted 64-bit unsigned integer. The caller must have permission to access the encrypted value.

```
decrypt (euint128)
```solidity
decrypt(euint128 value) → uint128
```
```

Decrypts an encrypted 128-bit unsigned integer. The caller must have permission to access the encrypted value.

```
decrypt (euint256)
```solidity
```

```
decrypt(euint256 value) → uint256
```
```

Decrypts an encrypted 256-bit unsigned integer. The caller must have permission to access the encrypted value.

```
decrypt (eaddress)
```solidity
decrypt(eaddress value) → address
```
```

Decrypts an encrypted Ethereum address. The caller must have permission to access the encrypted value.

### ### Decrypt Result Retrieval

```
getDecryptResult
```solidity
getDecryptResult(uint256 input) → uint256
```
```

Retrieves the decrypted result of a previously decrypted value. This function should be called after requesting decryption with `decrypt()`.

The function will revert if the decryption result is not available yet.

```
getDecryptResultSafe
```solidity
getDecryptResultSafe(uint256 input) → uint256 result, bool decrypted
```
```

Safely retrieves the decrypted result of a previously decrypted value. Unlike `getDecryptResult`, this function returns a boolean flag indicating whether the decryption is complete, avoiding the need to handle reverts.

Returns: `result`: The decrypted value, `decrypted`: A boolean indicating whether the decryption has completed successfully

### ### Access Control

```
allow (ebool)
```solidity
allow(ebool value, address account)
```
```

Grants permission to the specified account to access the encrypted boolean value.

```
allow (euint8)
```solidity
allow(euint8 value, address account)
```
```

Grants permission to the specified account to access the encrypted 8-

bit unsigned integer.

```
allow (euint16)
```solidity
allow(euint16 value, address account)
```
```

Grants permission to the specified account to access the encrypted 16-bit unsigned integer.

```
allow (euint32)
```solidity
allow(euint32 value, address account)
```
```

Grants permission to the specified account to access the encrypted 32-bit unsigned integer.

```
allow (euint64)
```solidity
allow(euint64 value, address account)
```
```

Grants permission to the specified account to access the encrypted 64-bit unsigned integer.

```
allow (euint128)
```solidity
allow(euint128 value, address account)
```
```

Grants permission to the specified account to access the encrypted 128-bit unsigned integer.

```
allow (euint256)
```solidity
allow(euint256 value, address account)
```
```

Grants permission to the specified account to access the encrypted 256-bit unsigned integer.

```
allow (eaddress)
```solidity
allow(eaddress value, address account)
```
```

Grants permission to the specified account to access the encrypted Ethereum address.

```
allowThis (ebool)
```solidity
allowThis(ebool value)
```
```

Grants permission to the current contract to access the encrypted boolean value.

```
allowThis (euint8)
```solidity
allowThis(euint8 value)
```
```

Grants permission to the current contract to access the encrypted 8-bit unsigned integer.

```
allowThis (euint16)
```solidity
allowThis(euint16 value)
```
```

Grants permission to the current contract to access the encrypted 16-bit unsigned integer.

```
allowThis (euint32)
```solidity
allowThis(euint32 value)
```
```

Grants permission to the current contract to access the encrypted 32-bit unsigned integer.

```
allowThis (euint64)
```solidity
allowThis(euint64 value)
```
```

Grants permission to the current contract to access the encrypted 64-bit unsigned integer.

```
allowThis (euint128)
```solidity
allowThis(euint128 value)
```
```

Grants permission to the current contract to access the encrypted 128-bit unsigned integer.

```
allowThis (euint256)
```solidity
allowThis(euint256 value)
```
```

Grants permission to the current contract to access the encrypted 256-bit unsigned integer.

```
allowThis (eaddress)
```solidity
allowThis(eaddress value)
```
```

Grants permission to the current contract to access the encrypted Ethereum address.

```
allowGlobal (ebool)
```solidity
allowGlobal(ebool value)
```
```

Grants global permission to access the encrypted boolean value.

```
allowGlobal (euint8)
```solidity
allowGlobal(euint8 value)
```
```

Grants global permission to access the encrypted 8-bit unsigned integer.

```
allowGlobal (euint16)
```solidity
allowGlobal(euint16 value)
```
```

Grants global permission to access the encrypted 16-bit unsigned integer.

```
allowGlobal (euint32)
```solidity
allowGlobal(euint32 value)
```
```

Grants global permission to access the encrypted 32-bit unsigned integer.

```
allowGlobal (euint64)
```solidity
allowGlobal(euint64 value)
```
```

Grants global permission to access the encrypted 64-bit unsigned integer.

```
allowGlobal (euint128)
```solidity
allowGlobal(euint128 value)
```
```

Grants global permission to access the encrypted 128-bit unsigned integer.

```
allowGlobal (euint256)
```solidity
allowGlobal(euint256 value)
```
```

Grants global permission to access the encrypted 256-bit unsigned integer.

```
allowGlobal (eaddress)
```solidity
```

```
allowGlobal(eaddress value)
```
```

Grants global permission to access the encrypted Ethereum address.

```
allowSender (ebool)
```solidity
allowSender(ebool value)
```
```

Grants permission to the message sender to access the encrypted boolean value.

```
allowSender (euint8)
```solidity
allowSender(euint8 value)
```
```

Grants permission to the message sender to access the encrypted 8-bit unsigned integer.

```
allowSender (euint16)
```solidity
allowSender(euint16 value)
```
```

Grants permission to the message sender to access the encrypted 16-bit unsigned integer.

```
allowSender (euint32)
```solidity
allowSender(euint32 value)
```
```

Grants permission to the message sender to access the encrypted 32-bit unsigned integer.

```
allowSender (euint64)
```solidity
allowSender(euint64 value)
```
```

Grants permission to the message sender to access the encrypted 64-bit unsigned integer.

```
allowSender (euint128)
```solidity
allowSender(euint128 value)
```
```

Grants permission to the message sender to access the encrypted 128-bit unsigned integer.

```
allowSender (euint256)
```solidity
allowSender(euint256 value)
```
```

Grants permission to the message sender to access the encrypted 256-bit unsigned integer.

```
allowSender (eaddress)
```solidity
allowSender(eaddress value)
```
```

Grants permission to the message sender to access the encrypted Ethereum address.

```
allowTransient (ebool)
```solidity
allowTransient(ebool value, address account)
```
```

Grants temporary permission to the specified account to access the encrypted boolean value.

```
allowTransient (euint8)
```solidity
allowTransient(euint8 value, address account)
```
```

Grants temporary permission to the specified account to access the encrypted 8-bit unsigned integer.

```
allowTransient (euint16)
```solidity
allowTransient(euint16 value, address account)
```
```

Grants temporary permission to the specified account to access the encrypted 16-bit unsigned integer.

```
allowTransient (euint32)
```solidity
allowTransient(euint32 value, address account)
```
```

Grants temporary permission to the specified account to access the encrypted 32-bit unsigned integer.

```
allowTransient (euint64)
```solidity
allowTransient(euint64 value, address account)
```
```

Grants temporary permission to the specified account to access the encrypted 64-bit unsigned integer.

```
allowTransient (euint128)
```solidity
allowTransient(euint128 value, address account)
```
```

Grants temporary permission to the specified account to access the

encrypted 128-bit unsigned integer.

```
allowTransient (euint256)
```solidity
allowTransient(euint256 value, address account)
```
```

Grants temporary permission to the specified account to access the encrypted 256-bit unsigned integer.

```
allowTransient (eaddress)
```solidity
allowTransient(eaddress value, address account)
```
```

Grants temporary permission to the specified account to access the encrypted Ethereum address.

```
isAllowed (ebool)
```solidity
isAllowed(ebool value, address account) → bool
```
```

Checks if the specified account has permission to access the encrypted boolean value.

```
isAllowed (euint8)
```solidity
isAllowed(euint8 value, address account) → bool
```
```

Checks if the specified account has permission to access the encrypted 8-bit unsigned integer.

```
isAllowed (euint16)
```solidity
isAllowed(euint16 value, address account) → bool
```
```

Checks if the specified account has permission to access the encrypted 16-bit unsigned integer.

```
isAllowed (euint32)
```solidity
isAllowed(euint32 value, address account) → bool
```
```

Checks if the specified account has permission to access the encrypted 32-bit unsigned integer.

```
isAllowed (euint64)
```solidity
isAllowed(euint64 value, address account) → bool
```
```

Checks if the specified account has permission to access the encrypted 64-bit unsigned integer.

```
isAllowed (euint128)
```

```
```solidity
```

```
isAllowed(euint128 value, address account) → bool
```

```
```
```

Checks if the specified account has permission to access the encrypted 128-bit unsigned integer.

```
isAllowed (euint256)
```

```
```solidity
```

```
isAllowed(euint256 value, address account) → bool
```

```
```
```

Checks if the specified account has permission to access the encrypted 256-bit unsigned integer.

```
isAllowed (eaddress)
```

```
```solidity
```

```
isAllowed(eaddress value, address account) → bool
```

```
```
```

Checks if the specified account has permission to access the encrypted Ethereum address.

## ## Bindings

The FHE library provides binding libraries that enable syntactic sugar for working with encrypted types. These bindings allow for more intuitive and object-oriented usage patterns.

## ### Available Binding Functions

With bindings, you can use encrypted types with dot notation:

```
```solidity
```

```
// Without bindings
```

```
euint8 sum = FHE.add(a, b);
```

```
// With bindings
```

```
euint8 sum = a.add(b);
```

```
```
```

Each encrypted type has a corresponding binding library that includes all the operations available for that type. For example, with `euint8` bindings:

## #### Arithmetic Operations

```
```solidity
```

```
// Use secure encrypted input
```

```
InEuint8 encryptedInputA; // This would be provided by the user's  
client-side encryption
```

```
InEuint8 encryptedInputB; // This would be provided by the user's
```

client-side encryption

```
euint8 a = FHE.asEuint8(encryptedInputA);
euint8 b = FHE.asEuint8(encryptedInputB);
```

```
euint8 sum = a.add(b);          // Addition
euint8 diff = a.sub(b);         // Subtraction
euint8 product = a.mul(b);      // Multiplication
euint8 quotient = a.div(b);     // Division
euint8 remainder = a.rem(b);    // Remainder
euint8 squared = a.square();    // Square
````
```

## #### Bitwise Operations

```
````solidity
euint8 bitwiseAnd = a.and(b);   // AND
euint8 bitwiseOr = a.or(b);     // OR
euint8 bitwiseXor = a.xor(b);   // XOR
euint8 bitwiseNot = a.not();     // NOT
euint8 shiftLeft = a.shl(b);    // Shift Left
euint8 shiftRight = a.shr(b);   // Shift Right
euint8 rotateLeft = a.rol(b);   // Rotate Left
euint8 rotateRight = a.ror(b);  // Rotate Right
````
```

## #### Comparison Operations

```
````solidity
ebool isEqual = a.eq(b);        // Equal
ebool isNotEqual = a.ne(b);     // Not Equal
ebool isLessThan = a.lt(b);     // Less Than
ebool isLessEqual = a.lte(b);   // Less Than or Equal
ebool isGreaterThan = a.gt(b);  // Greater Than
ebool isGreaterEqual = a.gte(b); // Greater Than or Equal
````
```

## #### Min/Max Functions

```
````solidity
euint8 minimum = a.min(b);      // Minimum
euint8 maximum = a.max(b);      // Maximum
````
```

## #### Type Conversion

```
````solidity
ebool converted = a.toBool();    // Convert to ebool
euint16 toU16 = a.toU16();      // Convert to euint16
euint32 toU32 = a.toU32();      // Convert to euint32
euint64 toU64 = a.toU64();      // Convert to euint64
euint128 toU128 = a.toU128();   // Convert to euint128
euint256 toU256 = a.toU256();   // Convert to euint256
````
```

```

Access Control
```solidity
a.decrypt(); // Decrypt
a.allow(address); // Allow access
a.allowThis(); // Allow this contract
a.allowGlobal(); // Allow global access
a.allowSender(); // Allow sender
a.allowTransient(address); // Allow transient access
bool hasAccess = a.isAllowed(address); // Check access
```

```

## ## Security Considerations

1. **Initialization**: All FHE functions check if their inputs are initialized and set them to 0 if not.
3. **Decryption**: Decryption functions reveal the plaintext values and should be used with caution.
4. **Security Zones**: Some functions accept a `securityZone` parameter to isolate different encrypted computations. FHE operations can only be performed between ciphertexts that share the same security zone.
5. **Access Control**: The library provides fine-grained access control through the `allow*` functions.

## ## Example Usage

These examples show how to perform private computations (if  $b > a$  then sum else product) while keeping all intermediate values encrypted.

```

Basic Usage
```solidity
// Use secure encrypted input
InEuint8 encryptedInputA; // This would be provided by the user's
client-side encryption
InEuint8 encryptedInputB; // This would be provided by the user's
client-side encryption

euint8 a = FHE.asEuint8(encryptedInputA);
euint8 b = FHE.asEuint8(encryptedInputB);

// Perform operations
euint8 sum = FHE.add(a, b); // Encrypted addition
euint8 product = FHE.mul(a, b); // Encrypted multiplication
ebool isGreater = FHE.gt(b, a); // Encrypted comparison

// Conditional logic

```

```

euint8 result = FHE.select(isGreater, sum, product);

// Decrypt the result (allow the contract to access it)
result.allowThis();
uint8 decryptedResult = FHE.decrypt(result);
```

With Bindings
```solidity
// Use secure encrypted input
InEuint8 encryptedInputA; // This would be provided by the user's
client-side encryption
InEuint8 encryptedInputB; // This would be provided by the user's
client-side encryption

euint8 a = FHE.asEuint8(encryptedInputA);
euint8 b = FHE.asEuint8(encryptedInputB);

// Perform operations using dot notation
euint8 sum = a.add(b); // Encrypted addition
euint8 product = a.mul(b); // Encrypted multiplication
ebool isGreater = b.gt(a); // Encrypted comparison

// Conditional logic
euint8 result = isGreater.select(sum, product);

// Decrypt the result (allow the contract to access it)
result.allowThis();
uint8 decryptedResult = FHE.decrypt(result);
```

```

```

title: ACL Usage Examples
sidebar_position: 6

```

## # ACL Usage Examples

See [ACL Mechanism](../fhe-library/acl-mechanism.md) for explanation of why is the ACL mechanism needed.

### ### Solidity API

1. `FHE.allowThis(CIPHERTEXT\_HANDLE)` – allows the current contract access to the handle.
2. `FHE.allow(CIPHERTEXT\_HANDLE, ADDRESS)` – allows the specified address access to the handle.
3. `FHE.allowTransient(CIPHERTEXT\_HANDLE, ADDRESS)` – allows the specified address access to the handle for the duration of the transaction.

### ### Automatic tx-scoped Allowance

The contract that creates the value for the first time will automatically get ownership of the ciphertext **\*\*for the duration of the transaction\*\***,

by using ``ACL.allowTransient(this)`` behind the scenes.

```
```solidity
```

```
// Contract A
```

```
function doAdd(InEuint32 input1, InEuint32 input2) {
```

```
    euint32 handle1 = FHE.asEuint32(input1); // Contract A gets
```

```
temporary ownership of handle1
```

```
    euint32 handle2 = FHE.asEuint32(input2); // Contract A gets
```

```
temporary ownership of handle2
```

```
    euint32 result = FHE.add(handle1, handle2); // possible because
```

```
Contract A has ownership of handle1 and handle2
```

```
}
```

```
```
```

### ### Persistent Allowance for This Contract

To use the results in other transactions, explicit ownership must be granted with ``FHE.allow(address)`` or ``FHE.allowThis()``.

```
```solidity
```

```
contract A {
```

```
    private euint32 result;
```

```
    private euint32 handle1;
```

```
    function doAdd(InEuint32 input1, InEuint32 input2) {
```

```
        handle1 = FHE.asEuint32(input1);           // Contract A gets
```

```
temporary ownership of handle1
```

```
        euint32 handle2 = FHE.asEuint32(input2); // Contract A gets
```

```
temporary ownership of handle2
```

```
        result = FHE.add(handle1, handle2);        // Contract A gets
```

```
temporary ownership of result
```

```
        FHE.allowThis(result);                     // result is allowed
```

```
for future transactions
```

```
    }
```

```
    function doSomethingWithResult() {
```

```
        FHE.decrypt(result);           // Allowed
```

```
        FHE.add(handle1, result); // ACLNotAllowed (handle1 is not  
owned persistently)
```

```
    }
```

```
}
```

```
```
```

### ### Allowance for Decryptions

To decrypt a ciphertext off-chain via the decryption network, the issuer must be allowed on the ciphertext handle via

``FHE.allow(userAddress)``.

```

```solidity
contract A {
    private mapping(address -> uint256) balances;

    function transfer(InEuint32 _amount, address to) {
        euint32 amount = FHE.asEuint32(_amount);

        balances[msg.sender] = balances[msg.sender] - amount;
        balances[to] = balances[to] + amount;

        FHE.allow(balances[msg.sender], msg.sender); // now the sender
can decrypt her balance
        FHE.allow(balances[to], to); // now the
receiver can decrypt his balance

        // enable balance manipulation for future transactions
        FHE.allowThis(balances[msg.sender]);
        FHE.allowThis(balances[to]);
    }
}
```

```

### Allow other contracts

You can also allow other contracts to use your ciphertexts, either persistently or only for the course of this transaction via `FHE.allowTransient(handle, address)`.

```

```solidity
contract A {
    function doAdd(InEuint32 input1) {
        handle1 = FHE.asEuint32(input1); // Contract A gets
temporary ownership of handle1

        FHE.allowTransient(handle1, addressB); // Contract B is
allowed to use handle1 in this transaction alone
        // or
        FHE.allow(handle1, addressB); // Contract B is
allowed to use handle1 forever

        IContractB(addressB).doSomethingWithHandle1(handle1);
    }
}
```

```

```

title: Migrating to CoFHE
sidebar_position: 3

```

# Migrating Your Contract to CoFHE – It's Easy!

Want to upgrade your Fhenix L2 contract to use CoFHE? Let's walk through it with a simple example – a privacy-enabled ERC20 token.

## ## The Key Change: Async Operations

The main difference is that CoFHE uses asynchronous operations under the hood. But don't worry – the core logic stays almost exactly the same! Let's look at a Wrapping ERC20 contract that:

- Lets users wrap public tokens into encrypted ones
- Enables private transfers between users
- Allows checking encrypted balances securely

## ## Key Migration Considerations

When migrating your contracts from Fhenix L2 to CoFHE, keep these important points in mind:

### ### 1. Add allows where necessary

In CoFHE, you need to explicitly allow the contract to use encrypted numbers that it will operate on during its lifetime. This is done using the ``FHE.allowThis()`` function:

```
```Solidify
  FHE.allowThis(_encBalances[msg.sender]);
```
```

### ### 2. Remove ``FHE.req()``

In Fhenix L2, you could use ``FHE.req()`` to enforce conditions on encrypted values, but this requires synchronous operation and also reveals some information about the encrypted value. In CoFHE, which uses symbolic execution and operates asynchronously, this approach needs to be reimaged. There are several ways to handle this:

1. **\*\*Conditional Operations\*\***: Instead of requiring conditions, implement logic that naturally enforces constraints

- Use select to conditionally process values:  
``FHE.select(condition, valueIfTrue, valueIfFalse)``

2. **\*\*Neutral Transformations\*\***: Apply operations that don't change values for valid inputs but neutralize invalid ones

- Add or subtract zero: ``value + FHE.asEuint(0)``
- Multiply by one: ``value * FHE.asEuint(1)``

This approach is compatible with asynchronous operations and preserves privacy.

### 3. Remove `FHE.sealoutput()`

Sealoutput will be available through Cofhejs only, make sure to allow the issuer of the permit, in order to be able to request sealoutput later. \

For more info see [permit management](/docs/devdocs/cofhejs/permits-management) and [sealing-unsealing](/docs/devdocs/cofhejs/sealing-unsealing)

```
```Solidity
    // Transfers an encrypted amount.
    function _transferImpl(address from, address to, uint128 amount)
internal returns (uint128) {
    // Make sure the sender has enough tokens.
    uint128 amountToSend =
FHE.select(amount.lte(_encBalances[from]), amount, FHE.asEuint128(0));
    // Add to the balance of `to` and subtract from the balance of
`from`.
    _encBalances[to] = _encBalances[to] + amountToSend;
    _encBalances[from] = _encBalances[from] - amountToSend;
    // diff-add
    // The addresses of the balances should have ownership on
their balance.
    // diff-add
    FHE.allow(_encBalances[from], from);
    // diff-add
    FHE.allow(_encBalances[to], to);
    return amountToSend;
}
// diff-remove
function balanceOfEncrypted(
// diff-remove
    address account, Permission memory auth
// diff-remove
) virtual public view onlyPermitted(auth, account) returns (string
memory) {
    // diff-remove
    return _encBalances[account].seal(auth.publicKey);
// diff-remove
}
```
```

Here's how it works:

## Original Fhenix L2 Contract – updated

First, let's look at the original Fhenix L2 contract:

```
```javascript
```

```

pragma solidity ^0.8.20;

import "@fhenixprotocol/contracts/access/Permissioned.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@fhenixprotocol/contracts/FHE.sol";

contract WrappingERC20 is ERC20, Permissioned {
    mapping(address => uint32) internal _encBalances;

    constructor(string memory name, string memory symbol) ERC20(name,
symbol) {
        _mint(msg.sender, 100 * 10 ** uint(decimals()));
    }

    function wrap(uint32 amount) public {
        // Make sure that the sender has enough of the public balance
        require(balanceOf(msg.sender) >= amount);
        // Burn public balance
        _burn(msg.sender, amount);

        // convert public amount to encrypted by encrypting it
        uint32 encryptedAmount = FHE.asEuint32(amount);
        // Add encrypted balance to his current balance
        _encBalances[msg.sender] = _encBalances[msg.sender] +
encryptedAmount;
        // diff-add
        // Allow the contract to operate on the encrypted balance for
future operations
        // diff-add
        FHE.allowThis(_encBalances[msg.sender]);
        // diff-add
        // Allow the users to to use decrypt\sealoutput on their
balances
        // diff-add
        FHE.allow(_encBalances[msg.sender], msg.sender);
    }

    function unwrap(InEuint32 memory amount) public {
        uint32 _amount = FHE.asEuint32(amount);
        // diff-remove
        // verify that our encrypted balance is greater or equal than
the requested amount
        // diff-remove
        FHE.req(_encBalances[msg.sender].gte(_amount));
        // diff-add
        // Using select to avoid leaking the result balance
        // diff-add
        _amount = FHE.select(_encBalances[msg.sender].gte(_amount),
_amount, FHE.asEuint(0));
    }
}

```

```

        // subtract amount from encrypted balance
        _encBalances[msg.sender] = _encBalances[msg.sender] - _amount;
        // diff-add
        // Allow the contract to operate on the encrypted balance for
future operations
        // diff-add
        FHE.allowThis(_encBalances[msg.sender]);
        // diff-add
        // Allow the users to to use decrypt\sealoutput on their
balances
        // diff-add
        FHE.allow(_encBalances[msg.sender], msg.sender);
        // add amount to caller's public balance by calling the `mint`
function
        _mint(msg.sender, FHE.decrypt(_amount));
    }

    function transferEncrypted(address to, InEuint32 calldata
encryptedAmount) public {
        euint32 amount = FHE.asEuint32(encryptedAmount);
        // diff-remove
        // Make sure the sender has enough tokens.
        // diff-remove
        FHE.req(amount.lte(_encBalances[msg.sender]));
        // diff-add
        // Using select to avoid leaking the result balance
        // diff-add
        amount = FHE.select(_encBalances[msg.sender].gte(amount),
amount, FHE.asEuint(0));
        // Add to the balance of `to` and subtract from the balance of
`from`.
        _encBalances[to] = _encBalances[to] + amount;
        _encBalances[msg.sender] = _encBalances[msg.sender] - amount;
        // diff-add
        // Allow the contract to operate on the encrypted balance for
future operations
        // diff-add
        FHE.allowThis(_encBalances[msg.sender]);
        // diff-add
        FHE.allowThis(_encBalances[to]);
        // diff-add
        // Allow the users to to use decrypt\sealoutput on their
balances
        // diff-add
        FHE.allow(_encBalances[msg.sender], msg.sender);
        // diff-add
        FHE.allow(_encBalances[to], to);
    }

    // diff-remove

```

```

    function getBalanceEncrypted(Permission calldata perm) public view
onlySender(perm) returns (uint256) {
    // diff-remove
        return FHE.decrypt(_encBalances[msg.sender]);
    // diff-remove
    }
}
```

```

In the above example since allowing the users on every step of the way, the users can use decrypt/sealoutput directly from cofhejs or using `fhe.decrypt` as above while listening on the event ``DecryptResult``.

## Frontend Fhenix.js to Cofhejs

The main difference between Fhenix.js and Cofhejs is that Cofhejs is asynchronous and uses symbolic execution. This means that the encrypted values are not revealed until the user requests a decryption.

### 1. Initialization

```

```javascript
// diff-remove
const fhenixClient = new fhenixjs.FhenixClient({ provider: provider })
// diff-add
await cofhejs.initializeWithEthers({
    // diff-add
    provider,
    // diff-add
    signer,
    // diff-add
    environment: 'LOCAL',
    // diff-add
})
```

```

### 2. Encrypting values

With Cofhejs the encryption is done asynchronously, for this reason we can provide a callback function to log the encryption state (read about it [\[here\]\(../cofhejs/encryption-operations.md\)](#))

```

```javascript
// diff-remove
const envValue = await fhenixClient.encrypt_uint128(value)
// diff-add
const logState = (state) => {
    // diff-add
}
```

```

```

 console.log(`Log Encrypt State :: ${state}`)
 // diff-add
 }
 // diff-add
 const encryptedValues = await cofhejs.encrypt(logState,
 [Encryptable.uint32(10n), Encryptable.uint64(20n)])
 ``

```

### ### 3. Permits and Unsealing

In Fhenix.js the permit system was tied to contract address. The application required to request a permit for each contract address. In addition, to unseal the value, the contract function needs to use the permit in order to seal the value.

In Cofhejs the permit system is tied to the user address (issuer). The user can use default permit or create a new one using the `createPermit` function (read about it [\[here\]\(../cofhejs/permits-management\)](#)). To unseal the value, the contract need to return the encrypted value handle and the sealing process done off-chain.

```

````javascript
// diff-remove
const getExtractedPermit = async (contractAddress: string) => {
    // diff-remove
    if (fhenixClient != null && provider != null) {
        // diff-remove
        try {
            // diff-remove
            let permit = await
fhenixjs.getPermit(contractAddress, provider)
            // diff-remove
            fhenixClient.storePermit(permit)
            // diff-remove
            return
fhenixClient.extractPermitPermission(permit)
            // diff-remove
        } catch (err) {
            // diff-remove
            console.log(err)
            // diff-remove
        }
        // diff-remove
    }
    // diff-remove
    return null
    // diff-remove
}

```

```

// diff-remove
const permit = await getExtractedPermit(CONTRACT_ADDRESS)
// diff-remove
const sealedValue = await contract.getSealedValue(permit)

// diff-add
// use default permit
// diff-add
const encryptedValueHandle = await contract.getEncryptedValue()
// diff-add
const unsealed = await cofhejs.unseal(encryptedValueHandle,
FheTypes.Uint32)
// diff-add
// or generate a new permit
// diff-add
const permit = await cofhejs.createPermit({
    // diff-add
    type: 'self',
    // diff-add
    issuer: wallet.address,
    // diff-add
})
// diff-add
const encryptedValueHandle = await contract.getEncryptedValue()
// diff-add
const unsealed = await cofhejs.unseal(encryptedValueHandle,
FheTypes.Uint32, permit.data.issuer, permit.data.getHash())
// diff-add
```

```

### ### 4. Decryption

In Fhenix L2 the decryption was done on-chain using the `FHE.decrypt` function.

With Fhenix Co-Processor the decryption can be done in two ways:

1. Using `FHE.decrypt` in your solidity contract.
2. Off-chain using the `cofhejs.decrypt` function.

```

```javascript
// diff-add
const encryptedValueHandle = await contract.getEncryptedValue()
// diff-add
const decrypted = await cofhejs.decrypt(encryptedValueHandle,
FheTypes.Uint32)
```

```

```

title: Adding FHE to an Existing Contract
sidebar_position: 3

```

## # Adding FHE to an Existing Contract

Lets start with an existing contract, and walk through the steps to migrate the contract to CoFHE.

The contract is very simple voting contract

```
```solidity
// SPDX-License-Identifier: MIT
// OpenZeppelin Contracts (last updated v5.1.0) (token/ERC20/
ERC20.sol)

pragma solidity ^0.8.25;

contract VotingExample {
    struct Option {
        string name;
        uint64 votes;
    }

    struct Proposal {
        string name;
        uint256 deadline;
        Option[] options;
        mapping(address => bool) hasVoted;
        bool exists;
        uint8 winner;
    }

    address public owner;
    uint256 public proposalCount;
    mapping(uint256 => Proposal) public proposals;

    event ProposalCreated(
        uint256 indexed proposalId,
        string name,
        uint256 deadline
    );
    event VoteCast(
        uint256 indexed proposalId,
        address indexed voter,
        uint256 optionIndex
    );

    error NotOwner();
    error InvalidOptionCount();
    error ProposalNotFound();
    error DeadlineExpired();
    error AlreadyVoted();
}
```

```

error InvalidOptionIndex();

constructor() {
    owner = msg.sender;
}

modifier onlyOwner() {
    if (msg.sender != owner) revert NotOwner();
    _;
}

function createProposal(
    string memory _name,
    string[] memory _options,
    uint256 _deadline
) external onlyOwner returns (uint256) {
    if (_options.length < 2 || _options.length > 4)
        revert InvalidOptionCount();

    uint256 proposalId = proposalCount++;
    Proposal storage proposal = proposals[proposalId];

    proposal.name = _name;
    proposal.deadline = _deadline;
    proposal.exists = true;

    for (uint i = 0; i < _options.length; i++) {
        proposal.options.push(Option({name: _options[i], votes:
0}));
    }

    emit ProposalCreated(proposalId, _name, _deadline);
    return proposalId;
}

function vote(uint256 _proposalId, uint256 _optionIndex) external
{
    Proposal storage proposal = proposals[_proposalId];

    if (!proposal.exists) revert ProposalNotFound();
    if (block.timestamp >= proposal.deadline) revert
DeadlineExpired();
    if (proposal.hasVoted[msg.sender]) revert AlreadyVoted();
    if (_optionIndex >= proposal.options.length)
        revert InvalidOptionIndex();

    proposal.options[_optionIndex].votes++;
    proposal.hasVoted[msg.sender] = true;

    emit VoteCast(_proposalId, msg.sender, _optionIndex);
}

```

```

    }

    function getProposal(
        uint256 _proposalId
    )
        external
        view
        returns (
            string memory name,
            uint256 deadline,
            Option[] memory options,
            bool exists
        )
    {
        Proposal storage proposal = proposals[_proposalId];
        return (
            proposal.name,
            proposal.deadline,
            proposal.options,
            proposal.exists
        );
    }

    function hasVoted(
        uint256 _proposalId,
        address _voter
    ) external view returns (bool) {
        return proposals[_proposalId].hasVoted[_voter];
    }
}

```

In this contract, the owner can create a proposal with anywhere from 2–4 possible voting options. Users can then vote on this proposal by the deadline, at which point the proposal can be closed, and the result revealed.

Unfortunately, the contract is completely public, and the vote tallies can be observed during the voting period. We will update this contract to interact with CoFHE, and use FHE encrypted variables to store the votes until the result is revealed.

We will pay special attention to the following updates that need to be made:

1. ****Constant time computation.****
2. ****Handling the `if/else` case.****
3. ****Handling the `require` case.****
4. ****Revealing the result with `FHE.decrypt`.****

Migrating the Contract

1. The first thing that we need to do is add the `FHE.sol` import:

```
```diff
pragma solidity ^0.8.25;

+ import {FHE, uint64, InEuint64} from "@fhenixprotocol/cofhe-
contracts/FHE.sol";

contract VotingExample {
```
```

2. We next need to encrypt the amount of votes cast for each option, lets do this by replacing the `uint64 votes` with the encrypted version from `FHE.sol`:

```
```diff
 struct Option {
 string name;
- uint64 votes;
+ euint64 votes;
 }
```
```

3. Now that the votes type has changed, it must be initialized by performing a `trivialEncrypt` of the starting value:

```
```diff
function createProposal(
 string memory _name,
 string[] memory _options,
 uint256 _deadline
) external onlyOwner returns (uint256) {
 if (_options.length < 2 || _options.length > 4)
 revert InvalidOptionCount();

 uint256 proposalId = proposalCount++;
 Proposal storage proposal = proposals[proposalId];

 proposal.name = _name;
 proposal.deadline = _deadline;
 proposal.exists = true;

 for (uint i = 0; i < _options.length; i++) {
- proposal.options.push(Option({name: _options[i], votes:
0}));
+ proposal.options.push(
+ Option({name: _options[i], votes: FHE.asEuint64(0)})
+);
 }
}
```

```

 }

 emit ProposalCreated(proposalId, _name, _deadline);
 return proposalId;
...}

```

This will work, but it may make sense to prepare the trivially encrypted values in the constructor rather than in each transaction. Lets see how that would look:

```

```diff
contract VotingExample {
    ...

+   uint64 private EUINT64_ZERO;
+   uint64 private EUINT64_ONE;

    constructor() {
        owner = msg.sender;
+       EUINT64_ZERO = FHE.asEuint64(0);
+       EUINT64_ONE = FHE.asEuint64(1);
    }

    function createProposal(
        string memory _name,
        string[] memory _options,
        uint256 _deadline
    ) external onlyOwner returns (uint256) {

        ...

        for (uint i = 0; i < _options.length; i++) {
            proposal.options.push(
-               Option({name: _options[i], votes: FHE.asEuint64(0)})
+               Option({name: _options[i], votes: EUINT64_ZERO})
            );
        }

        ...
    }

...
}

```

4. We now need to handle the user's vote casting. The first thing that we need to do is hide which option the user is voting for. We can do this by replacing the `vote` function parameter `uint256 \_optionIndex` with `InEuint8 memory \_optionIndex`. `InEuint8` is an encrypted input type. We then need to convert the `InEuint8` to an

`euint8` for use in computation.

> NOTE:
Encrypting inputs requires the use of `[**cofhejs**]`(/docs/devdocs/quick-start/getting-started).
 Read more about `[**encrypted inputs**]`(../cofhejs/encryption-operations.md).

```
```diff
- function vote(uint256 _proposalId, uint256 _optionIndex) external
+ function vote(uint256 _proposalId, InEuint8 memory _optionIndex)
external {
+ euint8 optionIndex = FHE.asEuint8(_optionIndex);
```
```

5. We have made it to a special case: `**Constant time Computation**`. In order to preserve the confidentiality of the user's vote, we must make sure that we aren't leaking any information about the user's choice. If we only updated the voting option that the user has selected, then a user's vote could be deduced by simply watching which vote counter changes. Therefore, we must update `_all_` the vote counters:

```
```diff
function vote(uint256 _proposalId, InEuint8 memory _optionIndex)
external {
 euint8 optionIndex = FHE.asEuint8(_optionIndex);
 Proposal storage proposal = proposals[_proposalId];

 if (!proposal.exists) revert ProposalNotFound();
 if (block.timestamp >= proposal.deadline) revert
DeadlineExpired();
 if (proposal.hasVoted[msg.sender]) revert AlreadyVoted();
 if (_optionIndex >= proposal.options.length)
 revert InvalidOptionIndex();

- proposal.options[_optionIndex].votes++;
+ for (uint8 i = 0; i < proposal.options.length; i++) {
+ proposal.options[i].votes = FHE.add(
+ proposal.options[i].votes,
+ FHE.select(
+ _optionIndex.eq(FHE.asEuint8(i)),
+ EUINT64_ONE,
+ EUINT64_ZERO
+)
+);
+ }
 proposal.hasVoted[msg.sender] = true;

 emit VoteCast(_proposalId, msg.sender, _optionIndex);
}
```

```
...
```

Lets break down how this works:

- We iterate through each of the proposal options.
- We always perform an `FHE.add` on every options' vote count, this means that every options' vote count will change any time a user votes.
- We only want to increment the user's selected choice, so we use `FHE.select`.
- The api for select is `FHE.select(conditional, ifTrue, ifFalse)`.
- Without the FHE syntax, the logic is as follows:

```
```solidity
proposal.options[i].votes =
    proposal.options[i].votes +
    _optionIndex == i
    ? 1
    : 0;
...`
```

6. There is one more change to make to the `vote` function, which is to remove the `if/else` branch that relies on `_optionIndex`. This would also be the case if it was within a `require` statement.

```
```diff
- if (_optionIndex >= proposal.options.length)
- revert InvalidOptionIndex();
...`
```

> NOTE: It is important to never use an encrypted variable as part of a if/else branch, since the encrypted variable is always truthy. Instead, use `FHE.select` to replace the value with 0.

### 7. We need to do a few more things in the `vote` function before we are ready to move on. The first is to update the `VoteCast` event:

The event itself:

```
```diff
event VoteCast(
    uint256 indexed proposalId,
    address indexed voter,
-    uint256 optionIndex
+    uint8 optionIndex
);
...`
```

And its invocation:

```

```diff
- emit VoteCast(_proposalId, msg.sender, _optionIndex);
+ emit VoteCast(_proposalId, msg.sender, optionIndex);
```

```

This event will be emitted with the encrypted `euint8 optionIndex`, this is important as in the future the FHE block explorer will be able to decrypt these variables and show the true event log, but only if we make one more change:

8. Using `FHE.allow` and its variants to grant access to the variables (for both computation and decryption):

```

```diff
function vote(uint256 _proposalId, InEuint8 memory _optionIndex)
external {
 euint8 optionIndex = FHE.asEuint8(_optionIndex);
 Proposal storage proposal = proposals[_proposalId];

 if (!proposal.exists) revert ProposalNotFound();
 if (block.timestamp >= proposal.deadline) revert
DeadlineExpired();
 if (proposal.hasVoted[msg.sender]) revert AlreadyVoted();

 for (uint8 i = 0; i < proposal.options.length; i++) {
 proposal.options[i].votes = FHE.add(
 proposal.options[i].votes,
 FHE.select(
 optionIndex.eq(FHE.asEuint8(i)),
 FHE.asEuint64(1),
 FHE.asEuint64(0)
)
);
+ FHE.allowThis(proposal.options[i].votes);
 }
 proposal.hasVoted[msg.sender] = true;
+ FHE.allowSender(optionIndex);
 emit VoteCast(_proposalId, msg.sender, optionIndex);
}
```

```

> NOTE: It is critical to ensure that `FHE.allowThis` is used on encrypted variables that need to be used later in the contract's lifecycle. Contracts must have access to variables in order to perform FHE operations on those variables.

9. We now need to handle the finalization step.

Because the votes are encrypted for the lifetime of the proposal, after the proposal has ended, we need to decrypt the results and reveal the winner. Lets start by adding a `finalizeVote` function:

```
```solidity
function finalizeVote(uint256 _proposalId) external {
 if (msg.sender != owner) revert NotOwner();

 Proposal storage proposal = proposals[_proposalId];
 if (!proposal.exists) revert ProposalNotFound();
 if (block.timestamp < proposal.deadline) revert
DeadlineNotReached();

 for (uint8 i = 0; i < proposal.options.length; i++) {
 FHE.decrypt(proposal.options[i].votes);
 }
}
```
```

As you can see, we perform a `FHE.decrypt` request on each of the proposal's options' vote counts. These decryption results are performed asynchronously, and the decrypted results are posted to the chain as soon as they are available.

10. Checking the results.

Finally, we can update our `getProposal` function to check the final state of the proposal:

```
```diff
function getProposal(
 uint256 _proposalId
)
 external
 view
 returns (
 string memory name,
 uint256 deadline,
- Option[] memory options,
- bool exists
+ bool exists,
+ string[] memory options,
+ uint256[] memory votes,
+ bool finalized,
+ uint8 winner
)
{
 Proposal storage proposal = proposals[_proposalId];
- return (
```



In this tutorial, we walked through migrating a simple voting contract to use CoFHE. The key changes were:

1. Changing the vote counts from plain `uint64` to encrypted values using `euint64`
2. Modifying the voting function to use encrypted addition instead of plain addition
3. Updating the getter function to handle decryption of results safely

The resulting contract provides the same functionality as the original, but with the added privacy benefit that individual votes are not visible on-chain until the final tally is decrypted. This demonstrates how CoFHE can be used to add privacy to existing contracts with minimal changes to the core logic.

### Full FHE enabled Voting Example

```
```solidity
// SPDX-License-Identifier: MIT
// OpenZeppelin Contracts (last updated v5.1.0) (token/ERC20/
ERC20.sol)

pragma solidity ^0.8.25;

import {FHE, euint64, InEuint8} from "@fhenixprotocol/cofhe-contracts/
FHE.sol";

contract VotingExample {
    struct Option {
        string name;
        euint64 votes;
    }

    struct Proposal {
        string name;
        uint256 deadline;
        Option[] options;
        mapping(address => bool) hasVoted;
        bool exists;
        uint8 winner;
    }

    address public owner;
    uint256 public proposalCount;
    mapping(uint256 => Proposal) public proposals;

    euint64 private EUINT64_ZERO;
    euint64 private EUINT64_ONE;
```

```

event ProposalCreated(
    uint256 indexed proposalId,
    string name,
    uint256 deadline
);
event VoteCast(
    uint256 indexed proposalId,
    address indexed voter,
    uint256 optionIndex
);

error NotOwner();
error InvalidOptionCount();
error ProposalNotFound();
error DeadlineExpired();
error AlreadyVoted();
error InvalidOptionIndex();
error DeadlineNotReached();

constructor() {
    owner = msg.sender;
    EUINT64_ZERO = FHE.asEuint64(0);
    EUINT64_ONE = FHE.asEuint64(1);
}

modifier onlyOwner() {
    if (msg.sender != owner) revert NotOwner();
    _;
}

function createProposal(
    string memory _name,
    string[] memory _options,
    uint256 _deadline
) external onlyOwner returns (uint256) {
    if (_options.length < 2 || _options.length > 4)
        revert InvalidOptionCount();

    uint256 proposalId = proposalCount++;
    Proposal storage proposal = proposals[proposalId];

    proposal.name = _name;
    proposal.deadline = _deadline;
    proposal.exists = true;

    for (uint i = 0; i < _options.length; i++) {
        proposal.options.push(
            Option({name: _options[i], votes: FHE.asEuint64(0)})
        );
    }
}

```

```

        emit ProposalCreated(proposalId, _name, _deadline);
        return proposalId;
    }

    function vote(uint256 _proposalId, InEuint8 memory _optionIndex)
external {
        euint8 optionIndex = FHE.asEuint8(_optionIndex);
        Proposal storage proposal = proposals[_proposalId];

        if (!proposal.exists) revert ProposalNotFound();
        if (block.timestamp >= proposal.deadline) revert
DeadlineExpired();
        if (proposal.hasVoted[msg.sender]) revert AlreadyVoted();

        for (uint8 i = 0; i < proposal.options.length; i++) {
            proposal.options[i].votes = FHE.add(
                proposal.options[i].votes,
                FHE.select(
                    optionIndex.eq(FHE.asEuint8(i)),
                    EUINT64_ONE,
                    EUINT64_ZERO
                )
            );
            FHE.allowThis(proposal.options[i].votes);
        }
        proposal.hasVoted[msg.sender] = true;

        FHE.allowSender(optionIndex);
        emit VoteCast(_proposalId, msg.sender, optionIndex);
    }

    function finalizeVote(uint256 _proposalId) external {
        if (msg.sender != owner) revert NotOwner();

        Proposal storage proposal = proposals[_proposalId];
        if (!proposal.exists) revert ProposalNotFound();
        if (block.timestamp < proposal.deadline) revert
DeadlineNotReached();

        for (uint8 i = 0; i < proposal.options.length; i++) {
            FHE.decrypt(proposal.options[i].votes);
        }
    }

    function getProposal(
        uint256 _proposalId
    )
    external
    view

```

```

    returns (
        string memory name,
        uint256 deadline,
        bool exists,
        string[] memory options,
        uint256[] memory votes,
        bool finalized,
        uint8 winner
    )
}

Proposal storage proposal = proposals[_proposalId];

name = proposal.name;
deadline = proposal.deadline;
exists = proposal.exists;

options = new string[](proposal.options.length);
for (uint8 i = 0; i < proposal.options.length; i++) {
    options[i] = proposal.options[i].name;
}

votes = new uint64[](proposal.options.length);
finalized = true;
for (uint8 i = 0; i < proposal.options.length; i++) {
    (uint256 result, bool decrypted) =
FHE.getDecryptResultSafe(
    proposal.options[i].votes
);
    votes[i] = decrypted ? result : 0;
    if (!decrypted) finalized = false;
}

if (finalized) {
    uint256 maxVotes = 0;
    winner = 0;
    for (uint8 i = 0; i < proposal.options.length; i++) {
        if (proposal.options[i].votes > maxVotes) {
            maxVotes = proposal.options[i].votes;
            winner = i;
        }
    }
}

}

function hasVoted(
    uint256 _proposalId,
    address _voter
) external view returns (bool) {
    return proposals[_proposalId].hasVoted[_voter];
}

```

}

...